Data Structures and Amortized Analysis

A data structure turns abstract operations into maintained states. First specify legal states and operation guarantees; then ask how much work an entire operation sequence requires.

Read **representations and contracts → amortized analysis → hashing → search trees and augmentation → B-trees → Fibonacci heaps → integer universes**.

Prerequisites are specifications and invariants from Chapter 1, recurrence analysis from Chapter 2, and probability and heaps from Chapter 3. Two stacks and a resizable array introduce the cost tools needed later.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The same operation name can hide different contracts

An **abstract data type (ADT)** specifies observable behavior; arrays, links, and trees implement it. Complexity depends on the contract.

Operation	Question to settle first
DELETE	Is the argument a key, an array index, or an existing node handle?
SEARCH	Are duplicate keys allowed? Which record is returned?
POP / DEQUEUE	Is an empty call rejected, given a special result, or forbidden?
Pointer update	Must node identity and external handles remain valid?

Deleting a known doubly linked-list node can take constant time. Finding a node by key and then deleting it also incurs search cost. Every later bound uses its own argument and representation contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circular queue: physical position is not logical order

Fix capacity $C \geq 1$ and use array $A[0..C-1]$, front index h, and size s. A valid state satisfies $0 \leq h < C$ and $0 \leq s \leq C$. Logical item j is

$$Q[j] = A[(h + j) \bmod C], \qquad 0 \leq j < s.$$

Empty means $s = 0$; full means $s = C$. Do not infer occupancy from equal indices. The next insertion position is $(h + s) \bmod C$, writable only when not full. All physical slots are usable.

Old values left in the array are not necessarily queue members. Clearing a removed slot may release resources, but h, s determine membership. Fixed-capacity ENQUEUE and DEQUEUE take worst-case $O(1)$, excluding allocation and nonconstant element-copy costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Check boundaries before changing queue state

```
ENQUEUE(x):
    if s == C: return overflow
    A[(h+s) mod C] = x
    s = s + 1

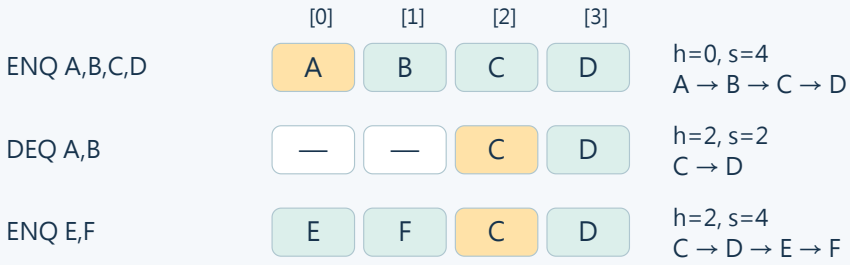
DEQUEUE():
    if s == 0: return underflow
    x = A[h]
    h = (h+1) mod C
    s = s - 1
    return x
```

Assume successful element reads/writes and sequential execution. A rejected call changes neither metadata nor live records. Distinguish error status from a returned data value. This is not a concurrent lock-free queue; sharing it requires synchronization.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

After wrapping, dequeue still returns C, D, E, F

Circular queue: physical slots and logical order



Amber = front; — = not live, even if old bits remain

Enqueue A–D, dequeue A and B, then enqueue E and F. Storage is [E,F,C,D], but $h = 2, s = 4$: read $2 \to 3 \to 0 \to 1$. Reject G; the next-insertion index equals h because the queue is full.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why circular indexing preserves FIFO

Before insertion, $s < C$. Position $(h + s) \bmod C$ differs from every live position $(h + j) \bmod C$, $0 \le j < s$: equality would imply $C \mid (s - j)$, contradicting $1 \le s - j < C$. Thus insertion overwrites no live record and appends exactly at the logical rear.

Dequeue returns $A[h] = Q[0]$. With $h' = (h + 1) \bmod C$ and $s' = s - 1$, each $0 \le j < s'$ satisfies

$$A[(h' + j) \bmod C] = A[(h + j + 1) \bmod C] = Q[j + 1].$$

The remaining sequence is precisely the old sequence without its first item. The empty initial state is valid; successful calls preserve the invariant and failed calls preserve state. Induction covers every legal call sequence.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Another representation: reserve one slot instead of size

CLRS § 10.1 uses head for the first item and tail for the next insertion slot, reserving one slot. In zero-based notation, empty is $h = t$ and full is $(t + 1) \bmod C = h$, so usable capacity is $C - 1$.

Another convention places f immediately before the first item and r at the last item. Empty is $f = r$ and full is $(r + 1) \bmod C = f$. Convert by $h = (f + 1) \bmod C$ and $t = (r + 1) \bmod C$. Renaming variables alone does not preserve read/write order.

This chapter uses h, s and all C slots. The derived next-insertion index is $t = (h + s) \bmod C$; $t = h$ can mean empty or full, distinguished by s. A reserved slot, a count, and a full flag are different designs with different tests.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Singly linked insertion: preserve the old successor first

Given a valid node p, insert a detached new node x after it. Node identity is fixed and keys may repeat. Node x must not already belong to the list.

```
q = p.next
x.next = q
p.next = x
```

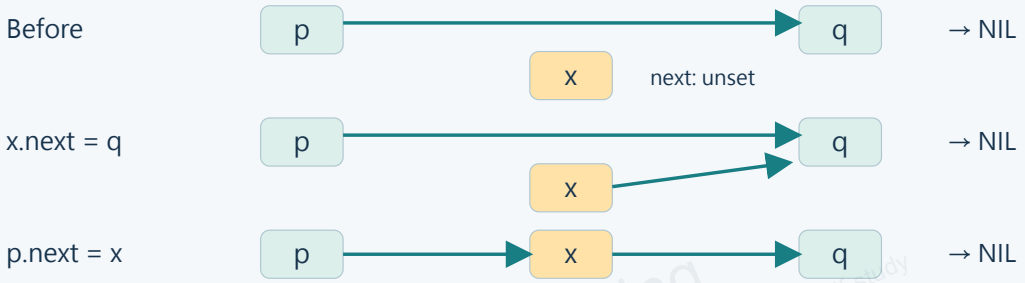
The path $p \to q$ becomes $p \to x \to q$; all other next links stay unchanged. The same code handles q=NIL. If a tail pointer is maintained and p was the tail, update tail=x.

Writing `p.next=x` before `x.next=p.next` creates a self-loop at x and can lose reachability to q. The $O(1)$ bound covers rewiring with known p, excluding search, allocation, and nonconstant data-copy costs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Three states: connecting a new node to the old path

Initialize the new node before changing the old path



Connect x.next to q before connecting p.next to x. In the intermediate state, x is not yet a member reachable from head. Node labels denote identity rather than key values.

Singly linked deletion: a node is not its predecessor

To delete the node after p, save `x=p.next`; reject x=NIL. Otherwise save `q=x.next`, then set `p.next=q`. If x was the tail, set tail=p. Reclaim x last. Given x alone without its predecessor, deleting a tail generally requires searching from head, taking worst-case $\Theta(n)$.

Copying the successor's data into x and deleting that successor changes which node owns which record and fails at the tail. It does not implement deletion with the node-identity contract used here.

In-place reversal preserves access to the untouched suffix

Reverse an acyclic, NIL-terminated singly linked list:

```
prev = NIL; cur = head
while cur != NIL:
    nxt = cur.next
    cur.next = prev
    prev = cur
    cur = nxt
head = prev
```

If a tail is stored, save old_head first and set tail=old_head at the end. During the loop, prev is the reversed processed prefix and cur begins the untouched suffix. Saving nxt preserves access after rewiring. Each iteration consumes one node: $\Theta(n)$ time and $O(1)$ extra space.

Circular doubly linked list: a sentinel joins the endpoints

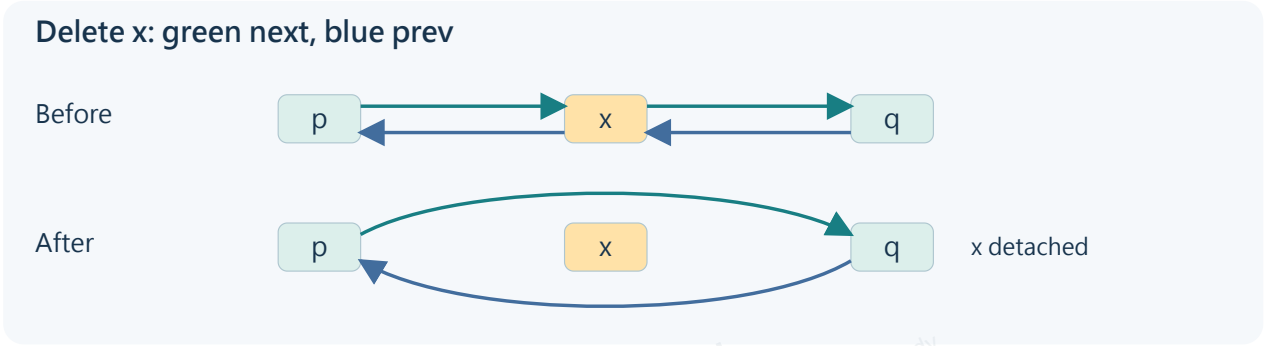
Allocate a permanent dummy node z, the sentinel, excluded from the data count. Empty means `z.next=z.prev=z`; otherwise z.next is the first item and z.prev the last.

For every node v in the ring, including z, maintain `v.next.prev=v` and `v.prev.next=v`. Require a valid member handle and forbid deleting z. Validating an arbitrary foreign handle is not included in constant-time rewiring of a known member.

```
DELETE(x):                // x != z, valid member
    p = x.prev; q = x.next
    p.next = q
    q.prev = p
```

Either neighbor may be z, so the code handles first, last, and sole data nodes. Reclaim x only after rewiring; external handles to x then become invalid. These are sequential operations, not intermediate states safe for unsynchronized concurrent readers.

Doubly linked deletion changes two neighbor links



Replace p.next=x and q.prev=x with p.next=q and q.prev=p. Other members keep their adjacencies. Old fields inside x do not make it a member after removal.

For a sole data item, p=q=z. The two writes restore z.next=z and z.prev=z, exactly the empty state. Two pointer writes do not mean deletion by an arbitrary key takes two steps: finding that key can still scan the entire list.

Doubly linked insertion must establish four links

Insert a detached x after a known p; p may be z. First save q=p.next:

```
x.prev = p
x.next = q
q.prev = x
p.next = x
```

The result is p↔x↔q, with all other links unchanged. For an empty list p=q=z, both z.next and z.prev become x. If p was last, q=z and z.prev automatically becomes the new last node.

Each new adjacency has its matching backlink, restoring the invariant. This order establishes correctness after a sequential call, not concurrent publication safety. A sentinel simplifies boundaries at the cost of one extra node per list; it does not improve search asymptotically.

Array indices as node handles: location and order differ

Parallel arrays key, next, and prev represent nodes without native pointers. Three fields at the same index belong to one object. Reserve index 0 for NIL and use indices 1–5 for objects.

Index	State	key	next	prev
1	free	—	4	—
2	live	B	3	5
3	live	C	0	2
4	free	—	0	—
5	live	A	2	0

With head=5, follow $5 \to 2 \to 3 \to 0$, giving A→B→C. With free=1, unused slots follow $1 \to 4 \to 0$. Adjacent physical indices need not be logical neighbors.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Allocation and reclamation keep live and free lists disjoint

Pool slots have three disjoint ownership classes: live, free, and caller-owned. ALLOCATE transfers a free slot to the caller; initialization and linking make it live. The free list uses only next and behaves as a stack.

```
ALLOCATE():
    if free == 0: return exhausted
    x = free; free = next[x]
    return x                    // initialize before linking

FREE(x):                        // detached, not already free
    next[x] = free
    free = x
```

Both operations change constantly many fields and take $O(1)$ in a fixed pool. Growing the pool and allocating variable-length records are outside this bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Object-pool trace: allocation through reclamation

Use the same five slots; head remains 5:

Completed operation	Live chain (NIL=0)	Free chain	Caller-owned
Initial	5(A)→2(B)→3(C)→0	1→4→0	none
ALLOCATE returns 1	5→2→3→0	4→0	1
Append D after C	5→2→3→1(D)→0	4→0	none
Unlink and FREE(2)	5→3→1→0	2→4→0	none

Append: key[1]=D, prev[1]=3, next[1]=0, next[3]=1. Remove B: set next[5]=3 and prev[3]=5, then next[2]=4 and free=2.

Double-free corrupts the free list; a stale handle can refer to an unrelated reused object. An in-range index does not establish a valid lifetime.

How two stacks preserve FIFO

Use S for incoming records and T for outgoing records. Read each sequence in the stated direction:

$$Q = \mathrm{reverse}(T_{\mathrm{bottom}\to\mathrm{top}}) \parallel S_{\mathrm{bottom}\to\mathrm{top}}.$$

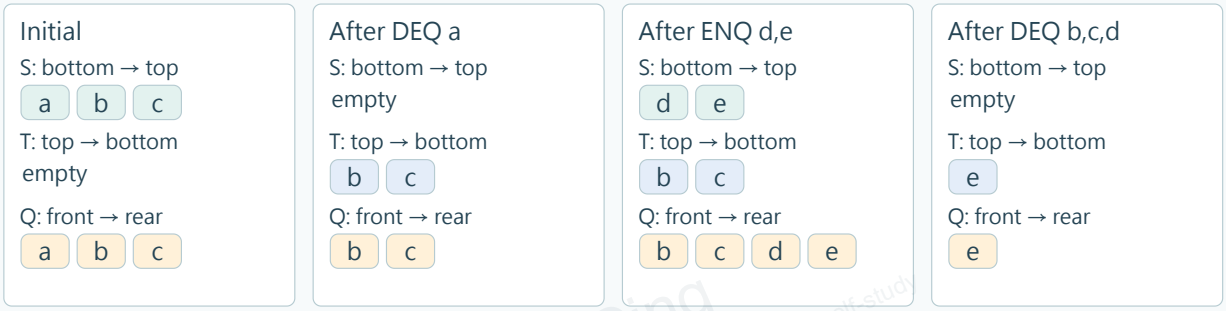
Here $\parallel$ denotes concatenation. **The top of T is the queue front.** Transfer all of S to T only when T is empty.

ENQUEUE pushes onto S, appending to Q. If T is nonempty, DEQUEUE pops its top, removing the first record of Q. If T is empty, transferring reverses S, making its oldest record the top of T while preserving queue order.

If both are empty, return underflow without changing state. Transferring before every dequeue would put new records ahead of older records still waiting in T.

Follow the order before counting the expensive transfer

One queue, two stacks: older records leave first



Read S bottom to top and T top to bottom. Enqueuing d, e leaves b, c waiting in T. Transfer S only after both b, c have left; then dequeue d.

One expensive call does not make every call expensive

Count one stack PUSH or POP as one unit, temporarily omitting fixed queue-call overhead. ENQUEUE costs 1. A DEQUEUE transferring k records costs $2k + 1$: pop and push each transferred record, then pop T.

Starting empty, each record can be **pushed onto S → popped from S → pushed onto T → popped from T** with each of these four primitives occurring at most once for that record. Thus e enqueues and d successful dequeues cost at most $3e + d$.

Any legal sequence of m operations has $e + d \leq m$, giving at most $3m$ primitive operations. Restoring fixed per-call overhead preserves $O(m)$. No distribution over sequences is assumed.

A single DEQUEUE can still cost $\Theta(n)$. A sequence guarantee is not a bound on each call's latency.

Potential: attach deferred work to the state

Operation i changes D_{i-1} into D_i at actual cost c_i. Choose a state function Φ and define

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

A positive $\Delta\Phi$ stores credit for future work; a negative change releases previously stored credit. Amortized cost is not a second measurement of execution time.

Summing a prefix of m operations cancels intermediate potentials:

$$\sum_{i=1}^{m} c_i = \sum_{i=1}^{m} \hat{c}_i + \Phi(D_0) - \Phi(D_m).$$

If all reachable states satisfy $\Phi(D_i) \geq \Phi(D_0)$, total amortized cost bounds total actual cost. A common sufficient condition is $\Phi(D_0) = 0$ and $\Phi \geq 0$.

Initial state and negative amortized charges matter

If only $\Phi \geq 0$ is known, without $\Phi(D_0) = 0$, the correct conclusion is

$$\sum c_i \leq \sum \hat{c}_i + \Phi(D_0).$$

A queue starting with n records in S may transfer all of them on its first dequeue. Either charge for constructing that initial state or retain an $O(n)$ initial term.

An individual $\hat{c}_i$ may be negative. The telescoping identity remains valid under the endpoint condition: the operation releases more credit than the actual work it performs.

An **amortized** bound controls total cost over legal sequences. An **expected** bound needs a probability model. An expected amortized bound can combine both; it must state both quantifiers.

Queue potential counts records not yet transferred

Let $\Phi(S, T) = 2|S|$. Both stacks start empty, so $\Phi_0 = 0$.

Operation	Actual c	$\Delta\Phi$	$\hat{c}$
ENQUEUE	1	2	3
DEQUEUE with nonempty T	1	0	1
Transfer $k \geq 1$ records, then dequeue	$2k + 1$	$-2k$	1
Report underflow on empty queue	0	0	0

The last row counts stack primitives only. Including queue-call overhead adds a fixed constant to each row. Records in T have already paid their transfer cost.

Every prefix has total cost at most $3e + d$, agreeing with the aggregate argument that each record incurs at most four stack primitives.

A complete ledger for interleaved operations

Both stacks run **bottom to top**; $\Phi = 2|S|$. Both totals are 19 because endpoint potentials are zero.

Operation	S	T	c	Φ	$\hat{c}$
ENQ a	a	empty	1	2	3
ENQ b	a,b	empty	1	4	3
ENQ c	a,b,c	empty	1	6	3
DEQ → a	empty	c,b	7	0	1
ENQ d	d	c,b	1	2	3
ENQ e	d,e	c,b	1	4	3
DEQ → b	d,e	c	1	4	1
DEQ → c	d,e	empty	1	4	1
DEQ → d	empty	e	5	0	1

Accounting: every prefix must pay its own bill

Operation i costs c_i but is charged a_i. The bank balance is

$$B_t = B_0 + \sum_{i=1}^{t} (a_i - c_i).$$

Start empty with $B_0 = 0$. Proving $B_t \geq 0$ for every prefix of every legal sequence gives $\sum_{i=1}^{t} c_i \leq \sum_{i=1}^{t} a_i$.

Borrowing against future operations is insufficient. The caller may stop while the bank is overdrawn. An initial credit $B_0 > 0$ must appear in the total-cost bound; it is not free work.

Attach credit to objects to show that it cannot be spent twice. If total credit depends only on the current state, setting $\Phi(D) = B(D)$ recovers the potential method.

MULTIPOP: who pays for a batch of removals?

An initially empty stack supports PUSH, POP, and MULTIPOP(k), with $k \geq 0$. The last operation removes at most k items and stops when empty. Each successful push or pop costs one unit.

Charge PUSH 2: pay 1 now and leave 1 on the new item. Charge POP and MULTIPOP 0: removed items pay with their stored credit. **Every item still on the stack carries exactly one unit.** Induction proves the bank never goes negative.

Among m calls, let p be the number of pushes and q the number of successful removals. Since $q \leq p$, actual cost is $p + q \leq 2p \leq 2m$. Pushing the same object again is a new insertion requiring new payment.

This model counts successful item operations. Adding one unit of checking/dispatch per API call changes the bound to $m + p + q \leq 3m$; an empty MULTIPOP does not literally take zero time.

Binary counter: fix width and overflow semantics

Use a fixed width $k \geq 1$. Bit $A[0]$ is least significant; the value is $\sum_{j=0}^{k-1} A[j]2^j$. Start at zero. INCREMENT adds one modulo 2^k, so all ones wraps to all zeros.

```
INCREMENT(A, k):
    j = 0
    while j < k and A[j] == 1:
        A[j] = 0
        j = j + 1
    if j < k:
        A[j] = 1
```

Cost c_i counts flipped bits. Tests and loop control take $\Theta(1 + c_i)$ actual time, so a constant overhead per call preserves the asymptotic sequence bound. Arbitrary-precision integers that increase their width require a separate representation contract.

A carry does not flip every bit on every call

Carry: trailing ones and the next zero

Amber = flipped on this call; most significant bit first

	A[3]	A[2]	A[1]	A[0]	
0110 → 0111	0	1	1	1	c = 1 ΔΦ = +1 ĉ = 2
0111 → 1000	1	0	0	0	c = 4 ΔΦ = -2 ĉ = 2
1000 → 1001	1	0	0	1	c = 1 ΔΦ = +1 ĉ = 2

The transition 0111→1000 flips four bits but also spends stored potential. Next, distinguish an ordinary carry from overflow.

Analyze ordinary carry and overflow separately

For a nonoverflowing carry through t trailing ones, clear those ones and set the next zero: $c = t + 1$. Let Φ be the number of set bits. Then $\Delta\Phi = 1 - t$ and $\hat{c} = 2$.

Overflow is a separate branch: $t = k$, every bit clears and no bit is set. Thus $c = k$, $\Delta\Phi = -k$, and $\hat{c} = 0$. The formula $t + 1$ does not apply to this branch.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One increment sequence, three consistent analyses

Starting at zero, perform $m \geq 1$ increments. Bit j flips once every 2^j calls, including wraps. The exact total is

$$T(m, k) = \sum_{j=0}^{k-1} \left\lfloor \frac{m}{2^j} \right\rfloor < 2m.$$

Aggregate analysis sums flips per position. **Accounting** charges 2 per call: setting a zero pays 1 now and leaves 1 for its later clearing. Every set bit owns its credit. **Potential analysis** counts those set bits, charging 2 for an ordinary increment and 0 for a wrap.

For $k = 4, m = 8$, positions flip $8, 4, 2, 1$ times, totaling 15 rather than $8 \times 4 = 32$. The proof covers every m; the numerical example checks the count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete ledger: from 0000 through wraparound

Four bits, initially $\Phi_0 = 0$. Bit strings are most significant first. The first eight charges total 16; subtract final potential 1 to get actual cost 15.

Call	Result	c_i	Φ_i	Cumulative cost
1	0001	1	1	1
2	0010	2	1	3
3	0011	1	2	4
4	0100	3	1	7
5	0101	1	2	8
6	0110	2	2	10
7	0111	1	3	11
8	1000	4	1	15

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete ledger: paying for the highest bit

Continue the same sequence. Calls 9–15 each charge 2; call 16 wraps and charges 0. Total actual and amortized costs both equal 30, with no credit remaining.

Call	Result	c_i	Φ_i	Cumulative cost
9	1001	1	2	16
10	1010	2	2	18
11	1011	1	3	19
12	1100	3	2	22
13	1101	1	3	23
14	1110	2	3	25
15	1111	1	4	26
16	0000	4	0	30

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Which changes invalidate the constant amortized bound?

A nonzero initial state: starting with k ones makes the first call cost k. The same potential only yields $\sum c_i \le 2m + \Phi_0 \le 2m + k$. Initial credit cannot be discarded.

Adding DECREMENT: for $k \ge 2$, increment from zero to $2^{k-1} - 1$, then alternate INCREMENT and DECREMENT. The states alternate between $0\underbrace{11\cdots1}_{k-1}$ and $1\underbrace{00\cdots0}_{k-1}$. Every alternating call flips all k bits.

Let preparation take $r = 2^{k-1} - 1$ calls, followed by $2r$ alternating calls. There are $m = 3r$ calls, and the alternating portion alone costs $2rk = \Omega(mk)$. The upper bound is $O(mk)$ since each call flips at most k bits, giving $\Theta(mk)$ on this family.

An expensive carry can now be immediately undone by a borrow. The previous credit attached to set bits is insufficient. Changing the supported operations requires a new cost proof.

Resizable arrays: specify what moving costs

State (n, M) records the number of live elements and capacity. Start at $(0, 0)$; allocate one slot on the first insertion, and double a full table before inserting.

Count **one new-record write plus the number of old records moved**. Temporarily omit fixed allocation, release, and test overhead. Initializing every newly allocated slot adds work and changes exact constants.

Insertion	Result (n, M)	Actual cost	Cumulative cost
1	(1,1)	1	1
2	(2,2)	2	3
3	(3,4)	3	6
4	(4,4)	1	7
5	(5,8)	5	12

Five insertions move $1 + 2 + 4 = 7$ old records. The moved-record count need not be smaller than the insertion count; the full geometric sum proves the linear total.

For growth alone, aggregate and potential bounds agree

After $m \geq 2$ insertions, let L be the largest power of two below m. Moves total $1 + 2 + \cdots + L = 2L - 1$, giving $T(m) = m + 2L - 1 < 3m$. Costs for $m = 0, 1$ are 0 and 1.

Choose $\Phi(0, 0) = 0$ and $\Phi(n, M) = 2n - M$ for nonempty tables. Reachable insertion-only states satisfy $M/2 < n \leq M$, so potential is nonnegative.

Branch	c	$\Delta\Phi$	$\hat{c}$
First insertion $(0, 0) \to (1, 1)$	1	1	2
Insert without moving	1	2	3
Full $(M, M) \to (M + 1, 2M)$	$M + 1$	$2 - M$	3

Hence $T(m) \leq 3m$. The first insertion has amortized cost 2; a formula involving $\log 0$ cannot be applied to the initial state.

Shrinking too eagerly repeatedly moves the same records

Consider the bad rule: double when full, and halve as soon as deletion leaves the table half full.

Let $k \geq 2$ be a power of two. Start full at (k, k). An insertion grows to $(k + 1, 2k)$ and moves k records. Deleting that record shrinks back to (k, k) and moves k again. Each repeated insert/delete pair moves $2k$ records and costs exactly $2k + 2$.

Building the initial state costs $O(k)$. Another $2r$ alternating operations cost $\Omega(kr)$. Taking $r = k$ gives $O(k)$ operations but $\Omega(k^2)$ total work.

Separate the thresholds: **grow only when full; shrink only when deletion leaves at most one-quarter occupancy**. Shrinking restores half occupancy, leaving room to accumulate credit before the opposite resize.

Resize contract: thresholds, empty tables, and record order

This example supports **APPEND and DELETE-LAST**. Deleting from the middle may shift a suffix; that cost is separate.

- The first APPEND allocates one slot. A full table doubles before writing the new record.
- DELETE-LAST removes the last record. If $M \geq 2$ and the resulting $n \leq M/4$, halve capacity and move the n survivors; otherwise retain capacity.
- Capacities are powers of two. Once allocated, minimum retained capacity is 1. An empty delete reports underflow without changing state.
- Each successful operation costs $1+$ records moved. Moving preserves survivor order but can invalidate external raw pointers.

Threshold convention: this example uses $n \leq M/4$. Textbook § 17.4 uses $n < M/4$, delaying contraction by one deletion when $M \geq 4$. Both allow constant amortized bounds, but their exact ledgers differ.

At most one resize occurs per operation. For $M \geq 4$, a shrink is triggered at exactly $n = M/4$: the prior state was above the threshold and one operation removes one record.

Follow the same records through growth and contraction

Separate resize thresholds leave time to save credit

Full	n=4, M=4	a b c d	cost —
APPEND e	n=5, M=8	a b c d e · · ·	cost 5 = 1 + 4
DELETE e	n=4, M=8	a b c d · · · ·	cost 1
DELETE d	n=3, M=8	a b c · · · · ·	cost 1
DELETE c	n=2, M=4	a b · ·	cost 3 = 1 + 2

Half full: retain capacity. Quarter full: halve capacity.

After growing to eight slots, retain capacity when four records remain. Shrink to four slots only when two remain. This separation between thresholds is **hysteresis**: it prevents immediate resize reversals.

Dense and sparse states store credit for different resizes

For $M \geq 2$, define

$$\Phi(n, M) = \begin{cases} 2n - M, & n \geq M/2, \\ M/2 - n, & n < M/2. \end{cases}$$

Also set $\Phi(0, 0) = \Phi(0, 1) = 0$ and $\Phi(1, 1) = 1$. Both branches equal zero at half occupancy; every reachable state has nonnegative potential.

Dense states accumulate credit during insertions to pay for growth. Sparse states accumulate credit during deletions to pay for contraction. A state uses exactly one branch, so credit is not counted twice.

The next page checks each branch. Empty-table cases at capacities 1 and 2 are separate because $M/4$ need not be an attainable integer count.

The general-capacity branches all have constant amortized bounds

Here M is capacity before the operation. Substitute into both branches when a nonresizing operation crosses half occupancy.

Case	c	$\Delta\Phi$	$\hat{c}$
Insert without growth	1	-1 or 2	0 or 3
Grow a full table	$M + 1$	$2 - M$	3
Delete without shrinking	1	-2 or 1	-1 or 2
Shrink after reaching $M/4$, $M \geq 4$	$1 + M/4$	$1 - M/4$	2

For the last row, before deletion $n = M/4 + 1$ and potential is $M/4 - 1$. Afterwards capacity is $M/2$ and count is $M/4$: exactly half full, with potential zero.

Example: $(3, 8) \to (2, 4)$ moves two records, so $c = 3$. Potential falls from 1 to 0, giving $\hat{c} = 2$.

Minimum-capacity cases and the total-cost conclusion

Check the remaining small-capacity boundaries separately:

State change	c	$\Delta\Phi$	$\hat{c}$
First $(0,0) \to (1,1)$	1	1	2
Reuse $(0,1) \to (1,1)$	1	1	2
Delete last $(1,1) \to (0,1)$	1	-1	0
Smallest shrink $(1,2) \to (0,1)$	1	0	1

Every successful operation has $\hat{c} \leq 3$. Initial potential is zero and final potential is nonnegative, so any m successful APPEND / DELETE-LAST operations cost at most $3m$ in this model.

Charging one unit for an underflow test adds an operation with unchanged potential and amortized cost 1. The bound does not cover arbitrary-position deletion, arbitrary shrinking policies, or unmodeled record-moving costs.

Dictionary: decide what inserting an existing key means

Maintain a key→record mapping with at most one record per key. INSERT updates the value of an existing key. DELETE takes a key and reports absence if missing. Equal hash values do not imply equal keys.

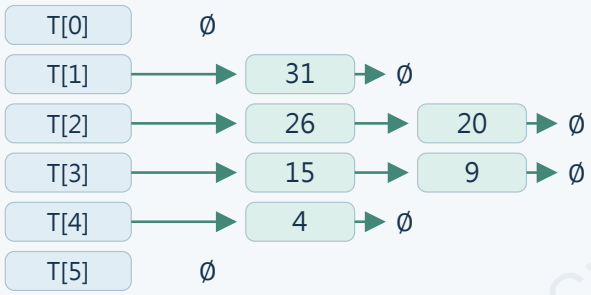
For universe $\{0, \ldots, M-1\}$, direct addressing uses the key as an index. Operations take worst-case $O(1)$, with $\Theta(M)$ space and ordinary full-table initialization. Hashing becomes useful when only $n \ll M$ records are present.

Use $h : U \to \{0, \ldots, m-1\}$. If $|U| > m$, every fixed h has collisions by the pigeonhole principle, so retain full keys for verification. Bounds assume keys, hashing, and equality take constantly many word operations. Long strings and arbitrary-precision integers require their own reading and arithmetic costs.

Chaining keeps colliding records in a list

m = 6; h(k) = k mod 6

Insert at heads: 4, 9, 15, 20, 26, 31



Find 20

26 → 20 ✓

Find 14

26 → 20 → ø

Same bucket: compare full keys

With $m = 6, h(k) = k \bmod 6$, insert 4,9,15,20,26,31 at chain heads. Slot 2 contains 26→20 and slot 3 contains 15→9. Searching for 20 checks two records; absent key 14 also checks both records in slot 2.

Constant-time head insertion does not cover all of INSERT

Let $\alpha = n/m$; chaining allows $\alpha > 1$. Search scans the designated chain until finding the key or reaching its end. Worst-case time is $\Theta(n)$.

Head insertion rewires links in $O(1)$ when the key is known to be new. Our update-on-existing-key INSERT must first search for duplicates. DELETE(key) also pays for search; only deletion through a valid doubly linked node handle supports direct $O(1)$ unlinking.

CLRS § 11.2 gives constant-time insertion and deletion under these different input contracts: insertion assumes a new object and deletion receives the object itself. The bounds are compatible once the contracts are distinguished.

Unsuccessful search: add the cost of each collision

First assume simple uniform hashing: fixed distinct keys have independent uniform bucket locations. For a fixed absent query q, define $I_x = \mathbf{1}[h(x) = h(q)]$. Then

$$L_q = \sum_{x \in K} I_x, \qquad E[L_q] = \sum_{x \in K} \frac{1}{m} = \frac{n}{m} = \alpha.$$

Search checks every record in that chain. Adding constant hashing and indexing work gives expected time $\Theta(1 + \alpha)$. At $\alpha = 0$, inspecting an empty bucket still costs time.

Linearity of expectation itself needs no independence between collision events. The full uniform model is sufficient; a universal family will later give an upper bound using only pairwise collision bounds.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Successful search averages records rather than buckets

Fix distinct insertion order $x_1, \ldots, x_n$, insert at heads, perform no deletions, and choose a successful query uniformly among stored records. Finding x_i checks itself and later inserted records in its bucket.

$$E[C_i] = 1 + \frac{n - i}{m}, \qquad E[C] = \frac{1}{n} \sum_{i=1}^{n} E[C_i] = 1 + \frac{n - 1}{2m}.$$

Adding constant hashing work gives $\Theta(1 + \alpha)$. Larger buckets contain more keys that may be queried; successful queries do not select buckets uniformly.

Our six-key table has two length-two chains and two singletons. Summed successful checks are $(1 + 2) + (1 + 2) + 1 + 1 = 8$, averaging $4/3$. This fixed-table average need not equal the random-model expectation $1 + 5/12$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Division and string hashing: a formula is not a distribution guarantee

Division hashing is $h(k) = k \bmod m$. For $m = 2^p$, it retains only p low bits: 0 and 8 both map to 0 when m=8. A prime modulus can avoid some input patterns, but does not prove uniformity for arbitrary key sets.

Horner evaluation uses $H_0 = 0$ and $H_{i+1} = (BH_i + c_i) \bmod m$. Induction shows this equals the full base-B integer modulo m. A length-ℓ string requires $\Theta(\ell)$ constant-cost steps.

With B=10, m=7, codes [1,2,3] yield intermediate values $1, 5, 4$, matching $123 \bmod 7 = 4$. If $B \equiv 1 \pmod m$, the result is just the code sum modulo m, unchanged by permutation. Computing the formula correctly does not establish low collision probability.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multiplication hashing: take high bits of the low word

Use $h(k) = \lfloor m \operatorname{frac}(kA) \rfloor$. For $A = s/2^w, m = 2^p$, compute $r_0 = ks \bmod 2^w$, then $h(k) = \lfloor r_0 / 2^{w-p} \rfloor$, with $1 \le p \le w$.

Write $ks = q2^w + r_0$. Then $kA = q + r_0/2^w$, so its fractional part is $r_0/2^w$. Multiplying by 2^p and flooring discards the low w−p bits of the low word, leaving its p high bits.

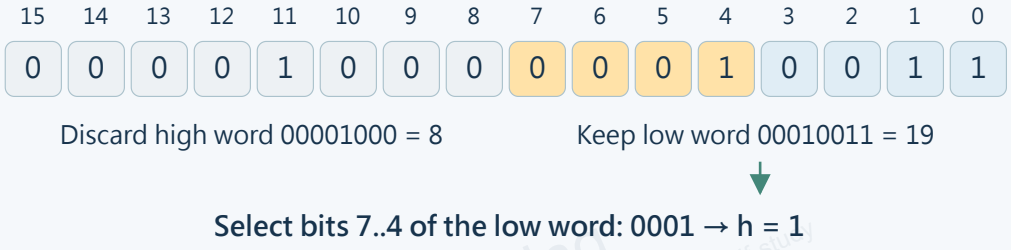
Next use w=8, s=159, k=13, p=4 to trace every bit; the result should be 1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multiplication example: from full product to slot index

w = 8; s = 159; k = 13; p = 4

ks = 2067 = 8 × 256 + 19



$2067 = 8 \cdot 256 + 19$. The low word is 00010011; its four high bits are 0001. Do not take the highest bits of the full product. Respect the stated word width.

Universal families randomize the choice of function

A finite family H is universal if every fixed pair of distinct keys x,y satisfies

$$\Pr_h[h(x) = h(y)] \leq \frac{1}{m}$$

when h is chosen uniformly from H. Keys and queries must be independent of that draw. No uniform input distribution or mutual independence of all collisions is required. An individual sampled table may still have a long chain.

Source convention: we use the “at most” definition of CLRS § 11.3.3. MIT 2005 Lecture 8 defines an exact collision probability and demonstrates a prime-modulus dot-product family. Its equality implies our inequality, but the reverse implication is invalid.

Universal chaining: the complete expected upper bound

Fix set K and query q. Let $Y_q = \sum_{x \in K, x \neq q} \mathbf{1}[h(x) = h(q)]$. Linearity and universality give

$$E[Y_q] \leq \frac{|K \setminus \{q\}|}{m}.$$

An absent query scans at most n/m records in expectation. A present query's entire chain has expected length at most $1 + (n-1)/m$, and search costs no more than scanning it. Thus each fixed query has expected time $O(1+\alpha)$ without requiring uniform successful queries.

For a fixed operation sequence with at most $O(m)$ live keys, update-style INSERT and DELETE(key) search first. Total expected time for t calls is $O(t)$ plus $\Theta(m)$ initialization. Queries chosen adaptively from h or earlier observations are not covered by this fixed-pair proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Affine family: prove the prime-field mapping first

Choose prime $P > \max U$, independently uniform $a \in \{1, \ldots, P-1\}$ and $b \in \{0, \ldots, P-1\}$, and define

$$h_{a,b}(k) = ((ak + b) \bmod P) \bmod m.$$

For fixed x≠y, let $r = (ax + b) \bmod P$ and $s = (ay + b) \bmod P$. Since $a(x-y) \not\equiv 0 \pmod P$, r≠s.

Conversely, each r≠s uniquely determines $a = (r-s)(x-y)^{-1} \bmod P$ and $b = r - ax \bmod P$. Uniform (a, b) therefore produces uniform ordered distinct (r, s) among $P(P-1)$ possibilities. Primality guarantees the inverse of nonzero x−y.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The final reduction still bounds collisions by 1/m

Condition on r. Each of the other P−1 residues s is equally likely. At most $\lceil P/m \rceil - 1 \leq (P-1)/m$ share r's residue modulo m. Hence

$$\Pr[h(x) = h(y) \mid r] \leq \frac{\lceil P/m \rceil - 1}{P-1} \leq \frac{1}{m}.$$

Averaging over r proves universality. For m≥P, reducing modulo m merges no distinct r,s, so collision probability is zero.

Write $P = qm + d, 0 \leq d < m$. If $r \bmod m < d$, its bucket has $q+1$ elements and q others after excluding r; otherwise a nonempty bucket has q elements and $q-1$ others. Thus the count is at most q; for $d = 0$ it is exactly $q-1$. Table size m need not be prime; P must be prime. Modular arithmetic must fit the assumed word model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why the MIT dot-product family also works

Let m be prime and represent keys by fixed-length vectors $x = (x_0, \ldots, x_r) \in \mathbb{F}_m^{r+1}$. Draw uniform coefficient vector a and use $h_a(x) = \sum_i a_i x_i \bmod m$.

For x≠y, choose j with $x_j - y_j \neq 0$. After fixing all other coefficients, collision requires

$$a_j(x_j - y_j) = -\sum_{i \neq j} a_i(x_i - y_i) \pmod m.$$

The difference is invertible, so exactly one of m values of a_j satisfies the equation. Collision probability is exactly 1/m. Here table modulus m must be prime, unlike the arbitrary table size of the affine family. Variable vector length also changes hashing cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Open addressing uses three distinct slot states

Store one record per slot, so n≤m. Each key has a reproducible fixed sequence $h(k,0),\ldots,h(k,m-1)$. To find space whenever available, these m positions must form a permutation of the table.

State	Search action	Deletion effect
OCCUPIED	Compare full keys	May become DELETED
DELETED (tombstone)	Continue probing	Not a stopping point
EMPTY	Stop: absent	Never created by ordinary deletion

EMPTY means unused since this table was built. An inserted key could not have skipped an EMPTY earlier in its sequence. Deletion leaves DELETED rather than breaking that path. A rebuild establishes a new table state.

Search and deletion probe at most m slots

```
FIND(k):
    for i = 0 .. m-1:
        j = h(k,i)
        if T[j] is EMPTY: return absent
        if T[j] is OCCUPIED and T[j].key == k:
            return j
    return absent
```

DELETE(k) runs FIND, changing a found slot to DELETED and otherwise preserving state. EMPTY and DELETED require separate tags rather than reserved values that could also be legal keys.

The loop bound matters: a table may contain tombstones but no EMPTY slots. An unsuccessful search must return after m probes rather than wait indefinitely for an empty slot.

Update-style INSERT remembers a tombstone but keeps searching

```
first = NONE
for i = 0 .. m-1:
    j = h(k,i)
    if T[j] is OCCUPIED and T[j].key == k:
        T[j].value = value; return j
    if T[j] is DELETED and first == NONE: first = j
    if T[j] is EMPTY:
        target = first if first != NONE else j
        T[target] = OCCUPIED(k,value); return target
if first != NONE: T[first] = OCCUPIED(k,value); return first
return full
```

An existing key may lie beyond the first DELETED. Inserting immediately could create duplicates. Establish absence at EMPTY or after all m probes, then reuse the saved slot. This assumes a full-table probe permutation.

Tombstones preserve search paths while allowing reuse

m = 7; home(k) = k mod 7

∅ = EMPTY; D = DELETED

	0	1	2	3	4	5	6	
Insert 1, 8, 15	∅	1	8	15	∅	∅	∅	Probes: 1 → 2 → 3
Delete 1	∅	D	8	15	∅	∅	∅	Find 15: 1 → 2 → 3
Update 8	∅	D	8	15	∅	∅	∅	Update value at slot 2
Insert 22	∅	22	8	15	∅	∅	∅	Probe to 4; write slot 1

Continue past D; rule out an existing key before reusing it.

Use $m = 7, h(k,i) = (k \bmod 7 + i) \bmod 7$. Keys 1,8,15 occupy slots 1,2,3. After deleting 1, finding 15 still follows $1 \to 2 \to 3$; a tombstone does not stop search.

A complete probe ledger for the same records

Three records remain. INSERT(22) confirms absence at slot 4, then writes slot 1.

Call	Slots probed	Write / result	Probes
INSERT(1,a)	1	store (1,a) at 1	1
INSERT(8,b)	1,2	store (8,b) at 2	2
INSERT(15,c)	1,2,3	store (15,c) at 3	3
DELETE(1)	1	slot 1=DELETED	1
FIND(15)	1,2,3	return slot 3	3
INSERT(8,b′)	1,2	update slot 2 only	2
INSERT(22,d)	1,2,3,4	reuse slot 1	4
FIND(29)	1,2,3,4	absent	4

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Probe formulas: first establish table coverage

Linear probing uses $(h_0(k) + i) \bmod m$ and covers every slot. Contiguous occupied runs attract further insertions, producing primary clustering.

Quadratic probing uses $(h_0(k) + c_1 i + c_2 i^2) \bmod m$ with constrained parameters. For m=7 and offset i^2, the first seven offsets are $0, 1, 4, 2, 2, 4, 1$, visiting only four slots. Insertion can fail despite unused slots elsewhere.

Equal home positions give identical sequences in these two schemes, the source of secondary clustering. Double hashing varies the step as well, but coverage still requires a step coprime to m; adding a function alone is not a proof.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two provable full-coverage probe schemes

Double hashing: use $h(k,i) = (h_1(k) + id) \bmod m$. Returning home requires $m \mid id$. With $g = \gcd(m,d)$, cancellation gives minimal positive period m/g. Full coverage therefore requires g=1. Use $1 \leq d < m$ for prime m, or odd d for power-of-two m.

Triangular offsets: for $m = 2^w$, use $f(i) = i(i+1)/2 \bmod m$, dividing as an integer first. For m=8 the offsets are $0, 1, 3, 6, 2, 7, 5, 4$.

If f(i)=f(j), then $2^{w+1} \mid (i-j)(i+j+1)$. The factors have opposite parity, so the even factor is divisible by 2^{w+1}. But $|i-j| < 2^w$ and $1 \leq i+j+1 < 2^{w+1}$. The only possibility is i−j=0. Thus f is injective and covers all m slots.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Uniform probing is a stronger idealized model

The following bounds assume each new key has a uniformly random permutation relative to the current occupied state, with no deletions or tombstones and n<m. This is sampling without replacement, not selecting a fresh random slot independently on every probe.

Linear, quadratic, and double hashing generate restricted sequence families. Uniform home locations or a universal family do not automatically imply the ideal distribution over m! permutations.

Separate collision probabilities for chaining, full-table probe coverage, and the distribution of probe permutations. The first two do not imply the expected formulas that follow.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Unsuccessful-search bound from tail probabilities

Let X count probes through the first EMPTY and $\alpha = n/m < 1$. The probability that the first i−1 positions are occupied is

$$\Pr(X \geq i) = \prod_{j=0}^{i-2} \frac{n-j}{m-j} \leq \alpha^{i-1}, \quad 1 \leq i \leq n+1.$$

For i=1 the empty product is 1; for i>n+1 the probability is zero. Each ratio is at most n/m since $m \geq n$. Using $X = \sum_{i \geq 1} \mathbf{1}[X \geq i]$,

$$E[X] = \sum_{i \geq 1} \Pr(X \geq i) \leq \sum_{i \geq 1} \alpha^{i-1} = \frac{1}{1-\alpha}.$$

The same bound covers insertion of a new key. At m=10,n=8 it is 5, not an exact count per query or an unconditional guarantee for linear probing.

Successful-search bound replays the insertion path

Fix n distinct insertions with no deletions or relocations. Choose successful queries uniformly among records. Insertion i+1 sees i occupied slots and expects at most $m/(m-i)$ probes; later searches replay that path.

$$E[C] \leq \frac{1}{n} \sum_{i=0}^{n-1} \frac{m}{m-i} = \frac{1}{\alpha} \sum_{k=m-n+1}^{m} \frac{1}{k}$$
$$\leq \frac{1}{\alpha} \int_{m-n}^{m} \frac{dx}{x} = \frac{1}{\alpha} \ln \frac{1}{1-\alpha}.$$

Since 1/x decreases, its integral over [k−1,k] is at least 1/k. No successful query exists at α=0; do not substitute zero into 1/α. The right side tends to 1 as α→0. Both the ideal probe model and uniform successful queries are needed for this formula.

Low live load can still require a full-table search

For m=7, fill all slots with seven distinct home positions, then delete every key. Now n=0 and α=0, but every slot is DELETED. Any absent query probes all seven positions.

The tombstone-free $1/(1-\alpha)$ bound is inapplicable. Tracking $\beta = (n+d)/m$, where d counts tombstones, exposes β=1 and the absence of an early EMPTY stopping point. β alone is not a performance proof for every implementation.

Rebuilding removes tombstones but costs allocation, rehashing, and reinsertion; open-address reinsertion itself requires probes. An expected amortized rebuild claim needs explicit thresholds, a random model, and rebuild rules, not just a promise to rehash periodically.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Perfect hashing moves random retries into construction

FKS stores a fixed set of n distinct keys. After construction, queries take worst-case $O(1)$ word operations with $O(n)$ space. Random retries give expected linear construction time, not worst-case linear time.

For n=0, return an empty dictionary without modulo zero. For n≥1 use m=n, draw a universal first-level h, and let bucket j contain n_j keys. Allocate n_j^2 secondary slots.

Let Y count unordered colliding pairs. Applying $n_j^2 = n_j + 2\binom{n_j}{2}$ to each bucket gives

$$\sum_j n_j^2 = n + 2Y, \qquad E\left[\sum_j n_j^2\right] \le n + 2\binom{n}{2}\frac{1}{n} = 2n - 1.$$

This controls total squared space through collision pairs; it does not require every bucket to be small.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

First level: check space before accepting the draw

Let $S = \sum_j n_j^2$. Markov gives $\Pr(S \geq 4n) \leq E[S]/(4n) < 1/2$. Independently redraw until the measured value satisfies S<4n.

Each trial clears n bucket counts and processes n keys in O(n). **Check S before allocating quadratic secondary tables for a rejected draw.** Acceptance probability exceeds 1/2, so expected trials are below 2 and first-level expected work is O(n).

Acceptance establishes a deterministic S<4n for the built structure. Merely knowing E[S]<2n would not establish that every output has linear size. Rare runs can require many retries; expected trial count is not a hard limit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Second level: squared space buys collision-free placement

Condition on an accepted first-level partition. A bucket of s≥2 keys gets s^2 slots and an independent universal h_j. Its collision-pair count Z satisfies

$$E[Z] \leq \binom{s}{2}\frac{1}{s^2} = \frac{s-1}{2s} < \frac{1}{2}, \qquad \Pr(Z \geq 1) \leq E[Z] < \frac{1}{2}.$$

Accept only after verifying zero collisions. Each trial clears s^2 slots and places s keys in $O(s^2)$, with fewer than two expected trials. Summing conditionally over the fixed accepted partition gives $O(\sum_j s_j^2) = O(n)$ expected work; averaging over the first level preserves O(n) total construction.

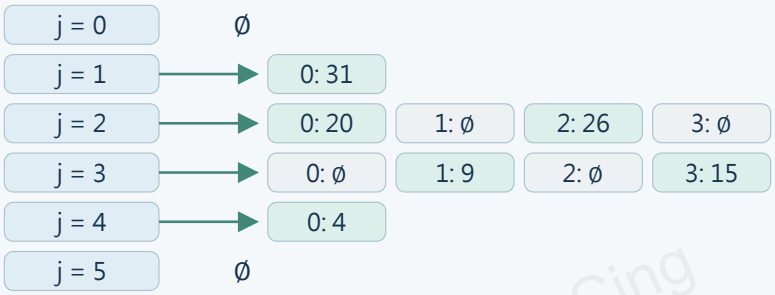
Empty buckets allocate nothing; singleton buckets store their record directly. Sizes, parameters, and pointers add O(n) space. Worst-case queries assume constant-cost hashing and key comparison; inserting new keys is outside this static contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two-level layout: place the same keys in actual slots

P = 37; h(k) = (k mod 37) mod 6

Two-key buckets: $h_j(k) = (k \bmod 37) \bmod 4$



26, 14 → j = 2 → slot 2

Stored key = 26

26 = 26 ✓

14 ≠ 26 ✗

Total secondary slots: 1 + 4 + 4 + 1 = 10

Use $K = \{4, 9, 15, 20, 26, 31\}$, P=37, and $h(k) = (k \bmod 37) \bmod 6$. Bucket sizes [0,1,2,2,1,0] have square sum 10<4n=24. Both two-key buckets use $h_j(k) = (k \bmod 37) \bmod 4$, occupying (0,2) and (1,3), respectively, with verified zero collisions.

An absent key can still reach an occupied perfect-hash slot

Query 26 follows first-level bucket 2 and secondary slot 2, reads key 26, and succeeds after equality comparison. Query 14 follows the same route but reads 26, so it is absent.

Query 5 reaches empty first-level bucket 5 and stops without evaluating a nonexistent secondary hash. Query 10 reaches singleton bucket 4 but reads 4≠10, so it also fails.

Perfect hashing separates stored keys; it does not certify membership for absent keys. Handle empty buckets, check slot occupancy, and compare full keys. After construction, present and absent queries both traverse constantly many levels, establishing the worst-case query bound.

BST constraints come from every ancestor

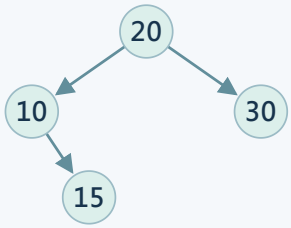
A binary search tree turns comparisons into branches. Continue the unique-key dictionary contract: a node stores key, value, left, right, and parent; missing links are NIL and root.parent=NIL. Inserting an existing key updates its value.

Every key in a node's entire left subtree is smaller; every key in its entire right subtree is larger. Parent-child comparisons alone are insufficient. A left turn tightens the upper bound; a right turn tightens the lower bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

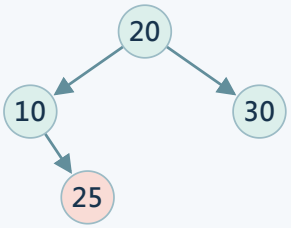
Ancestor intervals detect globally invalid placement

Every ancestor contributes a bound



15 ∈ (10, 20)

All bounds hold



25 > 10 ✓

25 < 20 ✗

Ancestor bound (10, 20)

CLRS § 12.1 also permits duplicates under non-strict inequalities. Here order is strict. Duplicates could instead use a count or a (key,unique-ID) ordering; these require an explicit contract.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Search rules out an entire subtree at each comparison

```
FIND(T,k):
    x = T.root
    while x != NIL:
        c = compare(k,x.key)   // one three-way comparison
        if c == 0: return x
        x = x.left if c < 0 else x.right
    return absent
```

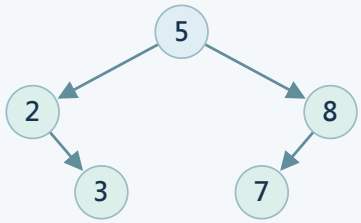
Invariant: if k exists, it lies in the subtree rooted at x. Global BST order rules out the other subtree after an unequal comparison. Each iteration descends one level; reaching NIL proves absence.

Depth d counts **edges** from the root, whose depth is 0. Successful search at depth d visits d+1 records. Failed search counts only visited non-NIL nodes. Count one three-way key comparison per visit; NIL tests and comparator instructions are separate costs.

Height h counts edges on a longest path; the empty tree has height −1. For nonempty trees search costs O(h+1); an empty tree costs O(1). A uniform bound is O(h+2).

Trace successful and unsuccessful searches in one tree

Root depth is 0; successful comparisons = depth + 1



d = 0

Find 7: 5 → 8 → 7; 3 comparisons

d = 1

Find 6: 5 → 8 → 7 → NIL

Failure also uses 3 comparisons

d = 2

Queries 9 and 10 fail to the right of 8

Finding 7 visits 5→8→7, using three key comparisons. Finding 6 visits those same three nodes before reaching NIL. Queries 9 and 10 visit 5→8→NIL, using two comparisons.

Count visited records for either outcome. The final NIL pointer test is not another key comparison.

Average cost depends on how queries are drawn

Uniform queries among the five stored keys give total successful comparisons 1+2+2+3+3=11, hence mean **11/5**. Uniform integer queries from 1 through 10 also include five failures.

query	1	4	6	9	10
keys visited	5,2	5,2,3	5,8,7	5,8	5,8
comparisons	2	3	3	2	2

The mixed mean is (11+12)/10=**2.3**. Gaps are not equally likely: $(8,+\infty)$ contains queries 9 and 10 here, with weight 2/10. Equal weights on NIL links would model a different experiment.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Inorder traversal and extrema

Inorder traversal lists left subtree, current node, then right subtree. Inductively, both subtrees produce increasing sequences. Global BST order places every left key below the root and every right key above it, so concatenation is sorted. Each node is processed once, giving $\Theta(n+1)$ time and $O(h+2)$ recursive stack space.

MINIMUM(x) requires $x \neq \text{NIL}$ and follows left links. If a left subtree exists it contains the minimum; otherwise x is smaller than everything to its right. MAXIMUM is symmetric. A dictionary-level minimum on an empty tree should return absent.

BST sorting includes construction. Inserting 1,2,…,n produces a right chain and costs $0+1+\dots+(n-1)=n(n-1)/2$ comparisons. Adding linear inorder traversal still gives $\Theta(n^2)$ total time.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Successor: descend right or climb to the first left turn

```
SUCCESSOR(x):                  // valid node handle
    if x.right != NIL: return MINIMUM(x.right)
    p = x.parent
    while p != NIL and x == p.right:
        x = p; p = p.parent
    return p                  // NIL means no successor
```

With a right subtree, its minimum is the next key. Without one, each climb from a right child passes a smaller ancestor; the first ancestor reached from its left side is the smallest larger key.

For root 5 with children 2,8, plus 2.right=3 and 8.left=7: succ(5)=7 via 8→7; succ(3)=5 via 3→2→5; succ(8)=NIL. PREDECESSOR swaps left and right.

A call follows one path and costs O(h+1). Starting at the minimum and repeatedly finding successors traverses each edge at most once downward and once upward, giving O(n) for the full sequence. Multiplying individual worst-case bounds does not prove Θ(nh).

Insertion finds a missing link, not an arbitrary leaf

```
INSERT(T,k,v):
    p = NIL; x = T.root
    while x != NIL:
        p = x; c = compare(k,x.key)
        if c == 0: x.value = v; return x
        x = x.left if c < 0 else x.right
    z = Node(k,v,NIL,NIL,p)
    if p == NIL: T.root = z
    elif k < p.key: p.left = z
    else: p.right = z
    return z
```

The search maintains the interval imposed by ancestors. At NIL, the new key satisfies every bound. The duplicate branch changes only the value, preserving key and identity. Arbitrarily changing a key in place can violate order.

Worst-case time is O(h+2), including empty-tree insertion, with O(1) new-node space.

Transplant changes the incoming parent link

```
TRANSPLANT(T,u,v):              // u is a valid member node
    p = u.parent
    if p == NIL: T.root = v
    elif u == p.left: p.left = v
    else: p.right = v
    if v != NIL: v.parent = p
```

This redirects the incoming link to u and updates a nonempty v's parent. **It does not reconnect v.left or v.right.** NIL is allowed as v but is not a writable parent-bearing node in this section.

If z has no left child, delete it with TRANSPLANT(T,z,z.right), including a leaf. If it has only a left child, transplant that child. Every retained subtree remains within the old ancestor bounds.

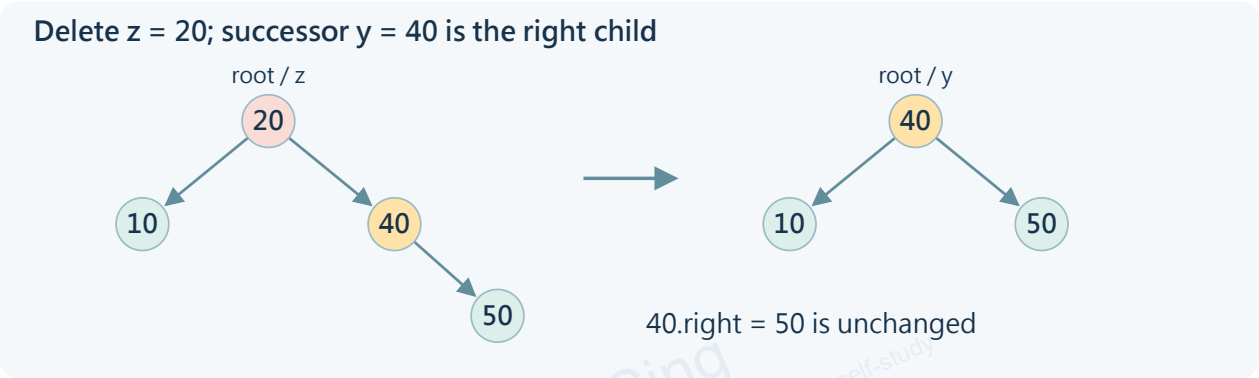
Deleting root 20 with sole right child 40 makes root=40 and 40.parent=NIL. Deleting the last node makes root=NIL. These cases change a constant number of links; DELETE(key) must also pay for finding z.

Two children: detach the successor before moving it

```
DELETE-NODE(T,z):
    if z.left == NIL: TRANSPLANT(T,z,z.right)
    elif z.right == NIL: TRANSPLANT(T,z,z.left)
    else:
        y = MINIMUM(z.right)
        if y.parent != z:
            TRANSPLANT(T,y,y.right)
            y.right = z.right; y.right.parent = y
        TRANSPLANT(T,z,y)
        y.left = z.left; y.left.parent = y
```

The right-subtree minimum y has no left child but **may have a right child**. A deeper y must first be replaced by y.right before receiving z's old right subtree. If y=z.right, leave its right link unchanged: assigning y.right=z.right would create y.right=y.

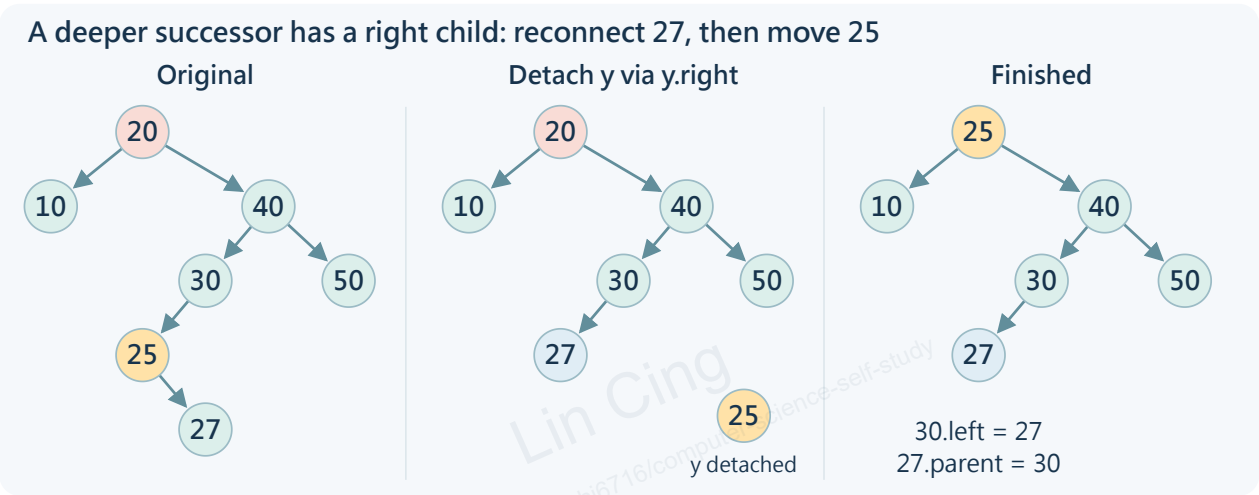
An immediate successor keeps its existing right child



Delete z=20: y=40 and y.parent=z. Redirect root to y and set y.left=10 and 10.parent=y. **The link y.right=50 never changes.**

Inorder changes from [10,20,40,50] to [10,40,50]. Every retained key, value, and object identity is unchanged; only z leaves the tree. For nonroot z, redirect its old parent's left or right link instead of root.

A deeper successor must not lose its right child



Delete root z=20. Its successor y=25 has right child 27. Writes are: ① 30.left=27 and 27.parent=30; ② y.right=40 and 40.parent=y; ③ root=y and y.parent=NIL; ④ y.left=10 and 10.parent=y.

Check order and reachability after a deep transplant

The middle panel shows y temporarily detached. Its stale links must be cleared or overwritten; the whole object pool is not a valid single BST during the update. Finally root=25,25.right=40,40.left=30,30.left=27.

Inorder [10,20,25,27,30,40,50] loses only 20. Writing 30.left=NIL would lose 27. Forgetting parent updates would break later upward successor searches.

Why is order preserved? Successor 25 is the minimum of the old right subtree. Every old left-subtree key is below 25, and every remaining right-subtree key is above 25. Key 27 lies between 25 and 30, so reconnecting it as 30.left is valid.

Every original object except z remains reachable exactly once. Finally check root.parent=NIL, matching child/parent links, and absence of cycles. Finding the successor costs O(h+1); rewiring is O(1). Reclaim z only after reconnection.

Deleting a key and deleting an object are different contracts

Let z.key=20 and its successor y.key=25. Both methods can preserve the same key set, but handles behave differently.

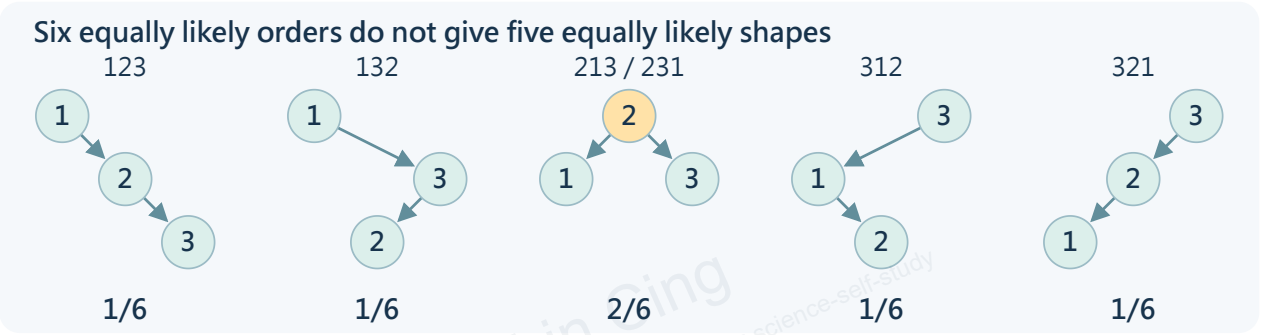
Method	Object occupying z's old position	Removed object	Handle effect
Copy y's payload, then delete y	Original z, now representing 25	y	z changes meaning; y is invalidated
Transplant node y	Original y in a new position	z	z is invalidated; y retains identity

Source convention: the handwritten notes illustrate copying; CLRS third edition § 12.3 transplants nodes, as we do here. Copying a dictionary record must move the matching value, not just its key. Extra fields, resource ownership, and external indices also need maintenance.

Either method may use the predecessor instead, with every direction mirrored consistently. Do not combine a predecessor diagram with successor code.

Uniform insertion orders do not give uniform tree shapes

Draw uniformly from n! orders on n distinct keys. The first key becomes root; given root rank r, each side retains uniform relative order. Analyze insertion-only construction.



Orders 213 and 231 share the balanced tree (2/6). Each other order gives a different shape (1/6). Sampling five shapes with probability 1/5 each is a different model.

Expected depth of a fixed rank: count ancestors

Write keys in order $x_1<\ldots<x_n$ and let D_i be the depth of x_i. For $j\neq i$, x_j is an ancestor of x_i **iff x_j is inserted first among the keys in the closed rank interval between i and j**.

Earlier keys outside the interval route both to the same side. If x_j appears first inside, it lies on the search path to x_i. Any other first key either is x_i or separates them. Thus

$$\Pr(x_j \text{ is an ancestor of } x_i) = \frac{1}{|j-i|+1}.$$

Depth counts ancestors. Linearity of expectation, without independent events, gives

$$E[D_i] = \sum_{j<i} \frac{1}{i-j+1} + \sum_{j>i} \frac{1}{j-i+1} = H_i + H_{n+1-i} - 2.$$

Here $H_t = \sum_{k=1}^{t} 1/k$. For n=5,i=3, expected depth is $2H_3 - 2 = 5/3$. A successful search adds one comparison, giving 8/3.

Expected construction cost sums all node depths

The nodes compared during insertion become exactly that node's ancestors in the finished tree; later insertions do not change its depth. Hence total construction comparisons are $C = \sum_i D_i$. Using

$$\sum_{i=1}^{n} H_i = \sum_{k=1}^{n} \frac{n-k+1}{k} = (n+1)H_n - n,$$

we obtain

$$E[C] = 2(n+1)H_n - 4n = \Theta(n \log n), \qquad n \geq 2.$$

Allocation and linking add Θ(n). Uniform successful search has expected mean comparisons E[C]/n+1. For n=3 these are 8/3 and 17/9 respectively; n=1 uses zero construction comparisons and one successful-search comparison.

This connects to quicksort using the first item as pivot recursively: comparison of a key pair depends on which key in their rank interval appears first. However, **logarithmic expected average depth does not yet bound expected maximum depth**.

Height analysis handles the maximum before expectation

Let X_n be edge height and $Y_n = 2^{X_n}$ for n≥1. Set **Y_0=0 as an auxiliary value**, not $2^{X_0} = 2^{-1}$. A singleton has Y_1=1. For n≥2, at least one subtree is nonempty; conditional on root rank R=r,

$$Y_n = 2 \max(Y_{r-1}^{L}, Y_{n-r}^{R}).$$

Write a_j=E[Y_j]. Conditional subtree orders are uniform. Since max(u,v)≤u+v,

$$a_n \leq \frac{2}{n} \sum_{r=1}^{n} (a_{r-1} + a_{n-r}) = \frac{4}{n} \sum_{j=0}^{n-1} a_j, \qquad n \geq 2.$$

This uses **conditional expectation**, not unconditional independence of random subtree sizes and R. It also avoids the false equality E[max(U,V)]=max(E[U],E[V]).

For example, $a_1 = 1$ and $a_2 = 2$. Among the six orders on three keys, two have height 1 and four have height 2, so a_3=(2 • 2+4 • 4)/6=10/3.

Bound exponential height, then return to height

Strong induction gives $a_n \le n^3$ for $n \ge 1$. The base is $a_1 = 1$. For $n \ge 2$, use $a_0 = 0$ and $\sum_{j=1}^{n-1} j^3 = [n(n-1)/2]^2$:

$$a_n \le \frac{4}{n} \sum_{j=1}^{n-1} j^3 = n(n-1)^2 \le n^3.$$

Let $\mu = E[X_n]$. Inequality $e^u \ge 1 + u$ implies $2^x \ge 2^\mu [1 + (\ln 2)(x - \mu)]$. Taking expectation cancels the linear term, yielding the needed Jensen inequality:

$$2^{E[X_n]} \le E[2^{X_n}] = a_n \le n^3 \quad \Longrightarrow \quad E[X_n] \le 3 \log_2 n.$$

The derivative of $e^u - 1 - u$ is $e^u - 1$, so its minimum is 0 at $u = 0$. This proves the tangent bound. Height has finite support, so these expectations exist.

A binary tree of edge height h contains at most $2^{h+1} - 1$ nodes; thus every tree has $h \ge \log_2(n+1) - 1$. Together the bounds give $\Theta(\log n)$ expected height. A single run may still have worst-case height $n - 1$.

Red-black trees control height through path invariants

A red-black tree is still a strictly ordered BST, with one extra color field. B means black and R means red. Figures include these labels as well as colors.

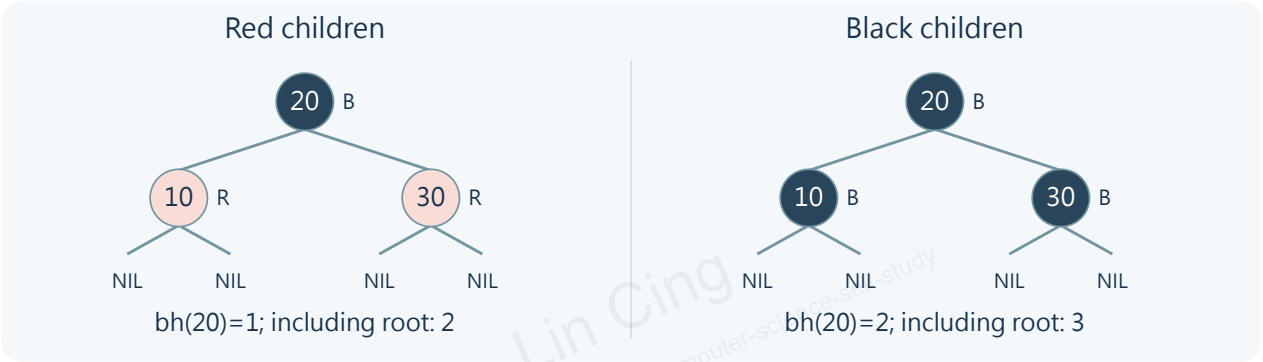
1. Each node is red or black, and the root is black.
2. Every external NIL position is black. A data node with no child still links to NIL.
3. Both children of a red node are black: consecutive red nodes are forbidden.
4. **From every node, all downward paths to NIL contain the same number of black nodes.** Checking only selected root–NIL paths is insufficient.

Use one shared sentinel with NIL.left=NIL.right=NIL, NIL.color=B, and root.parent=NIL. Require matching parent links only for non-NIL children. A duplicate-key insertion updates the value without changing shape or colors.

Earlier BST operations did not write NIL.parent. Deletion repair will temporarily use it for one deficient position. It cannot represent all missing-child parents simultaneously. Rotations still skip the parent write when β=NIL.

Fix exactly which nodes black-height counts

Define $bh(x)$ as the black nodes on a downward path from x to NIL, **excluding x and including the terminal NIL**; set $bh(NIL) = 0$. The path invariant makes this well-defined.



For a non-NIL child c, a red c has $bh(c) = bh(x)$; a black c has $bh(c) = bh(x) - 1$. The parent's count includes c, whereas c's own black-height excludes c. For c=NIL the relation is $bh(x) = 1 + bh(NIL)$.

Prove the height bound by counting required nodes

Retain internal edge height h: a singleton has h=0 and an empty tree h=−1. Separately let H count edges from root to external NIL. Then H=h+1 for a nonempty tree, and H=0 for an empty tree. Use H in the height proof without silently changing the meaning of h.

Lemma: The subtree at x contains at least $2^{bh(x)} - 1$ internal nodes. Induct on subtree height. For x=NIL, $0 = 2^0 - 1$. Otherwise each child has black-height at least bh(x)−1. Apply induction and count x:

$$n_x \geq 2\left(2^{bh(x)-1} - 1\right) + 1 = 2^{bh(x)} - 1.$$

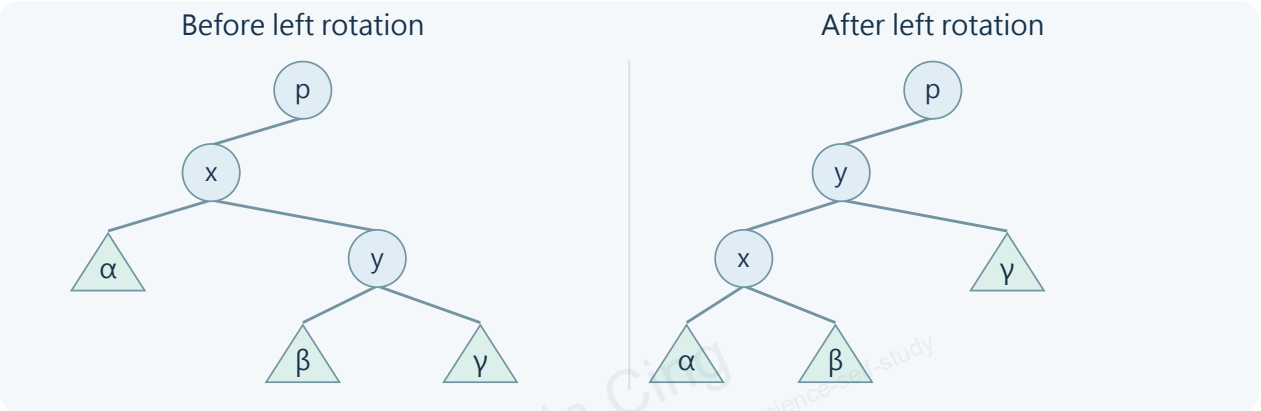
On a longest root–NIL path, let B count black nodes including both endpoints, and R count red nodes. Black endpoints and no adjacent reds imply $R \leq B - 1$. Its edge length is

$$H = B + R - 1 \leq 2(B - 1) = 2bh(root).$$

Hence $n \geq 2^{bh(root)} - 1 \geq 2^{H/2} - 1$ and $H \leq 2\log_2(n+1)$. The empty tree has H=0. Searching and its terminating test cost O(H+1), or uniformly O(log(n+2)), including the empty-tree constant cost.

This bounds search height; update repair still needs a termination and cost argument. Red-black trees do not require child heights to differ by at most one: that is an AVL condition.

Left rotation changes the parent of β and the child of p



Require x≠NIL and y=x.right≠NIL. The symbols α, β, γ denote entire subtrees, possibly NIL; p is x's **old parent**. Node y becomes the local root, and β moves from y.left to x.right.

Complete, symmetric pointer updates for both rotations

```
LEFT-ROTATE(T,x):                RIGHT-ROTATE(T,y):
    p=x.parent; y=x.right            p=y.parent; x=y.left
    beta=y.left                      beta=x.right
    x.right=beta                     y.left=beta
    if beta!=NIL: beta.parent=x      if beta!=NIL: beta.parent=y
    y.parent=p                       x.parent=p
    if p==NIL: T.root=y              if p==NIL: T.root=x
    elif p.left==x: p.left=y          elif p.left==y: p.left=x
    else: p.right=y                  else: p.right=x
    y.left=x                         x.right=y
    x.parent=y                       y.parent=x
```

Save the old parent p. Skip the sentinel.parent write for β=NIL. Require x.right≠NIL or y.left≠NIL, respectively.

Check each rotation write against a concrete tree

Take p=30, x=10, y=20, β=15, with p.left=x. Leave all other subtree contents unchanged.

Step	Write	Purpose
1	10.right=15	Reconnect β
2	15.parent=10	Update β's parent
3	20.parent=30	Connect the new local root upward
4	30.left=20	Replace the old outside child slot
5	20.left=10	Make the old root a left child
6	10.parent=20	Update the old root's parent

For a root rotation, step 3 becomes 20.parent=NIL and step 4 becomes T.root=20.

Intervals α<x<β<y<γ remain valid, preserving BST order and node identities. Keys and values do not change. **Rotation alone does not ensure red-black validity.**

Insert a red leaf and identify the only possible violation

Use the previous INSERT search. An existing key only updates its value. Insert a new node z as R with left=right=NIL and a correct parent link.

Replacing a black NIL by a red z with black NIL children preserves every affected path's black count. Inserting a black node would add one black only to the paths through that position.

At loop entry maintain: **z is red; if its parent is the root, that parent is black; the only possible violation is a red root or the red pair z and its parent.** BST order and equal black counts remain valid.

A black parent needs no repair. A new root is simply blackened at the end. A red parent p enters the loop and cannot be the root, so grandparent g exists and is black. The uncle u is g's other child; it may be NIL, whose color is safe to read.

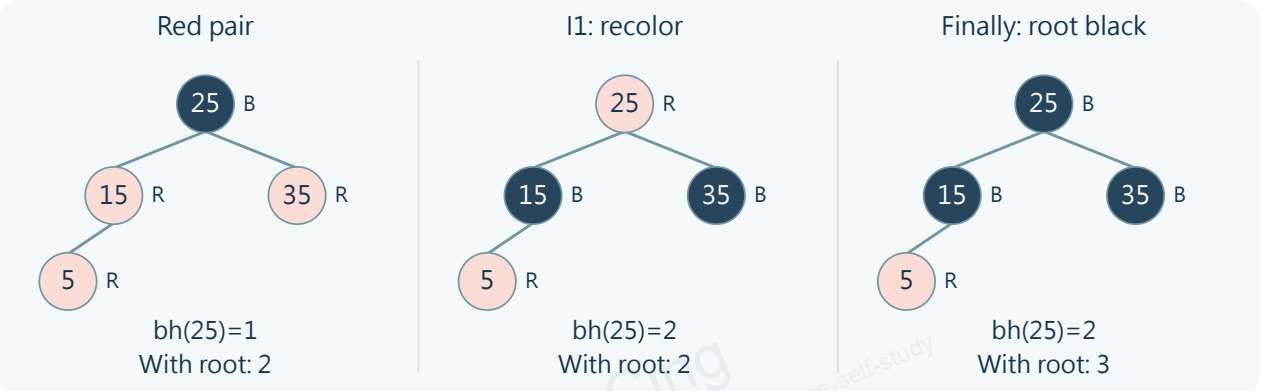
I2 is not an alternative terminating case: it falls through into I3. Rotations change parent links. I3 uses the current z.parent and grandparent, not cached pre-rotation p and g.

Insertion repair when the parent is a left child

```
while z.parent.color==R:
    if z.parent==z.parent.parent.left:
        u=z.parent.parent.right
        if u.color==R:                          // I1
            z.parent.color=B; u.color=B
            z.parent.parent.color=R
            z=z.parent.parent
        else:
            if z==z.parent.right:               // I2
                z=z.parent
                LEFT-ROTATE(T,z)
            z.parent.color=B                    // I3
            z.parent.parent.color=R
            RIGHT-ROTATE(T,z.parent.parent)
    else: apply the explicitly mirrored branch
T.root.color=B
```

Lin Cing

I1 preserves the black count seen by ancestors



Insert 25,15,35, then 5. The new 5R conflicts with parent 15R; uncle 35R is red too. Color 15 and 35 black, color 25 red, and move z to 25.

Lin Cing

I1: which black count changes?

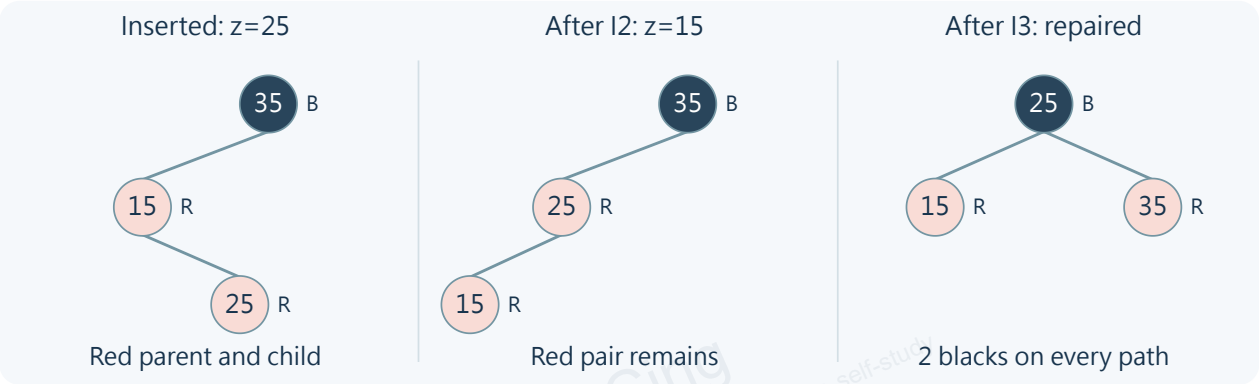
Let every path below p/u, excluding p/u, contribute b black nodes. A complete path through g originally contributes 1+b, and still contributes 1+b after g becomes red and p/u become black. **The ancestor-to-NIL black count through g stays unchanged**, although bh(g), which excludes g, rises from b to b+1.

Only g and its parent can now form a red pair. If g is the root, the final root-blackening step adds one black to all root–NIL paths. That final step must not be applied to a non-root g.

Counting range	Before I1	After I1
bh(g), excluding g	b	b+1
g through NIL, including g	1+b	1+b
Ancestor through g to NIL	outside count+1+b	outside count+1+b

For root g=25 and b=1, I1 leaves bh(25)=2 and a full-root count of 2. Final root blackening raises only the latter to 3.

I2 followed by I3: straighten the bend, then repair it

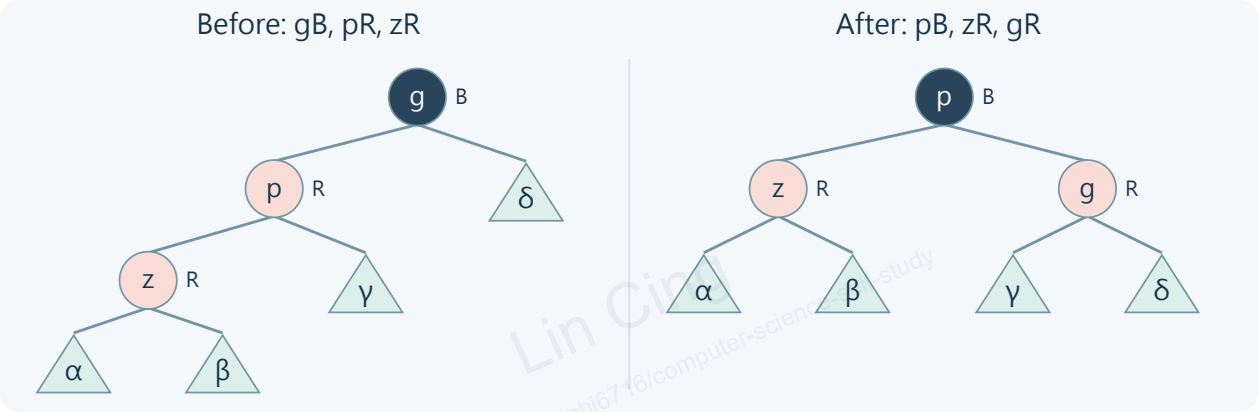


Insert 35,15,25. The uncle is black NIL; parent 15 is a left child and the new 25 is a right child. I2 first sets z to the old parent 15 and left-rotates 15. Now 25R is the parent of 15R: **the red pair still exists**.

I3 blackens the current parent 25, reddens grandparent 35, and right-rotates 35. Result: 25B with 15R,35R, and two blacks per path. Do not return after I2.

Why I3 preserves black counts and removes the red pair

Start with gB-left pR-left zR and a black uncle. Let α,β be z's children, γ be p's old right subtree, and δ be g's old right subtree. Blacken p, redden g, and right-rotate g.



I3: verify every repaired invariant

Red z forces α,β to have black roots; red p forces γ black; δ is the black uncle. Equal path counts make every boundary subtree contribute b blacks, including its root.

Destination subtree	Local black nodes before	Local black nodes after
α,β	gB, then b	pB, then b
γ	gB, then b	pB; gR adds zero; then b
δ	gB, then b	pB; gR adds zero; then b

All four destinations retain 1+b blacks. Each boundary subtree keeps its internal links and colors, preserving its own path equalities.

New local root p is black, so no outside red pair is possible. Red z has black children α,β; red g has black children γ,δ. An outer insertion chain skips I2; the same proof applies.

Write out the mirrored right-parent branch

This replaces the preceding code's else branch when z.parent is the grandparent's right child. The uncle is now on the left, and both rotation directions reverse.

```
u=z.parent.parent.left
if u.color==R:                                  // I1 mirrored
    z.parent.color=B; u.color=B
    z.parent.parent.color=R
    z=z.parent.parent
else:
    if z==z.parent.left:                         // I2 mirrored
        z=z.parent
        RIGHT-ROTATE(T,z)
    z.parent.color=B                            // I3 mirrored
    z.parent.parent.color=R
    LEFT-ROTATE(T,z.parent.parent)
```


Why insertion needs at most two rotations

I1 moves z upward by two levels without rotating, at most O(H) times. I2 falls through to I3, which blackens the local root and terminates. A black parent also terminates the loop; final root blackening resolves the remaining root-color case.

Branch	Primitive rotations	What follows
I1: red uncle	0	Move z up two levels; may repeat
I2: black uncle, inner child	1	Immediately perform I3
I3: black uncle, outer child	1	Black local root; terminate
Final root blackening	0	Done

The rotating suffix is either I3 or I2→I3. Repeated I1 steps add recolorings, not rotations.

Therefore there are at most two primitive rotations in total. Search plus recoloring costs O(log(n+2)); a new node uses O(1) space.

Two repair schedules must not be mixed

Comparison	Bottom-up here	Top-down 2-3-4 splitting
Two red children	Insert a red leaf, then repair upward	Split/recolor a 4-node during descent
Possible rotation	Black-uncle I2/I3	A red pair created during downward repair
Shared invariants	BST order, no red pair, equal black counts	Same goals, different repair-entry conditions

These algorithms may produce different valid shapes. Do not mix their recoloring schedules while retaining one algorithm's proof. A “double-rotation adjustment” may count as two primitive rotations; fix the unit and algorithm before comparing counts.

Coloring the edge to each red child gives a 2-3-4 interpretation after contracting red edges. General red-black trees allow either red-edge direction. Left-leaning red-black trees impose an additional convention.

Distinguish z, y, x, and y's original color

z is the record to remove. If z has fewer than two children, y=z; otherwise y is the minimum in z.right and moves as a whole object into z's position. x is the sole possible child filling **y's old position**, namely y.right for a successor with no left child; when y=z, use z's only child or NIL. Save yOriginalColor before giving moved y z.color. Missing black depends on yOriginalColor, not y's new color or merely z.color.

```
RB-TRANSPLANT(T,u,v):
    if u.parent==NIL: T.root=v
    elif u==u.parent.left: u.parent.left=v
    else: u.parent.right=v
    v.parent=u.parent              // write even when v==NIL
```

NIL.parent now records the parent of the current empty x position so repair can find its sibling. It is not a universal parent for all empty links. The root's parent is NIL. Rotations skip beta.parent for beta=NIL to preserve this context. After a stable operation, NIL.parent may be reset to NIL; NIL.color must always remain black.

RB-DELETE: relocate first, then inspect the original color

```
RB-DELETE(T,z):                  // z is an existing non-NIL record
    original=z.color
    if z.left==NIL:
        x=z.right; RB-TRANSPLANT(T,z,x)
    elif z.right==NIL:
        x=z.left; RB-TRANSPLANT(T,z,x)
    else:
        x,original=RELOCATE-SUCCESSOR(T,z)
    if original==B: DELETE-FIXUP(T,x)
    NIL.parent=NIL                // only after repair finishes
    reclaim z
```

Only object z is removed; all other identities and key/value pairs survive. A red original color removes no black; a black original color starts repair at replacement x. A key-based API must first search and return absent on failure. Never call this procedure on NIL.

Two children: preserve the successor's original color

```
RELOCATE-SUCCESSOR(T,z):          // requires two non-NIL children
    y=MINIMUM(z.right); original=y.color; x=y.right
    if y.parent==z:
        x.parent=y                // even when x==NIL
    else:
        RB-TRANSPLANT(T,y,x)
        y.right=z.right; y.right.parent=y
    RB-TRANSPLANT(T,z,y)
    y.left=z.left; y.left.parent=y
    y.color=z.color
    return x,original
```

Successor relocation: trace every changed link

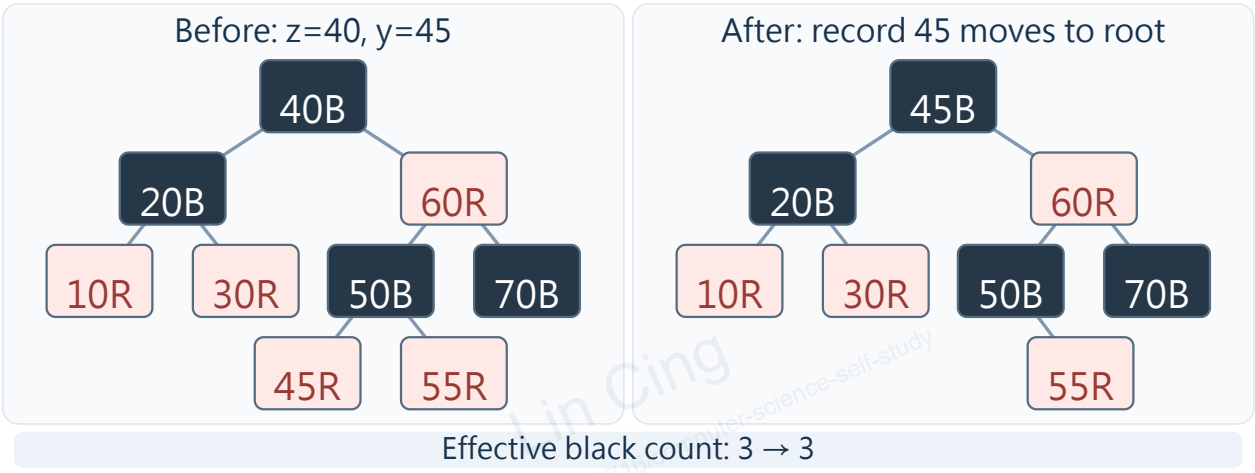
Insert 40,20,60,10,30,50,70,45,55. The tree has root 40B, left 20B(10R,30R), and right 60R with left 50B(45R,55R) and right 70B. Deleting z40 chooses y45R and x=NIL, with y.parent 50≠z.

Remove 45 from 50.left, giving 50.left=NIL and NIL.parent=50. Attach 60 as y.right and set 60.parent=45. Replace root 40 with y, then attach 20 as y.left and set 20.parent=45. Give y color B. Since y's original color was R, no black repair runs. Root is 45B, original 45's data moves with 45, and only record 40 is removed.

Successor y has no left child, so only y.right can fill its old position. If y is z's immediate right child, x's new parent is y, not the soon-reclaimed z. For a deeper y, the first transplant sets x.parent; moving y afterward must preserve that context.

Assigning y.color=z.color preserves the contribution at z's old position. The repair decision depends on **y's color before relocation**, a separate quantity.

A deeper successor: before and after relocation



Both panels show the same deletion: node 45 moves from the left of 50 to the root. Its old position is filled by x=NIL. The first transplant sets NIL.parent=50; moving 45 must preserve that context until repair or final cleanup resets NIL.parent=NIL. Omitted empty children are black NIL nodes.

Track a missing black at x without a third color

Removing an originally red y reduces no path black count. Giving moved y z's old color preserves the other local relationships, so no fix is needed. If y was black, temporarily assign its missing black to x's position. Count one extra black on each path through x to match the pre-deletion effective counts.

This proof-only extra black does not change the actual R/B color domain. A red x absorbs it by becoming black. At the root, drop it from all tree paths together. Otherwise the loop has a black nonroot x. Its sibling w cannot be NIL: w would contribute only one ordinary NIL black while x's side contributes at least NIL plus extra, contradicting equal effective black counts.

Below, x=p.left, w=p.right, near=w.left, and far=w.right. A right-side deficit exchanges left and right, giving near=w.right and far=w.left, and reverses rotations.

Figure legend: x +1 marks the proof-only extra black, not a third stored color. Unshown missing children are black NIL. Complete-path counts include the starting node and NIL.

The repair cursor tracks the unique extra-black position

```
DELETE-FIXUP(T,x):
    while x!=T.root and x.color==B:
        if x==x.parent.left: x=REPAIR-LEFT(T,x)
        else: x=REPAIR-RIGHT(T,x)
    x.color=B
```

The helpers return the next repair position: D2 returns the parent; D4 returns the root to terminate. Final blackening absorbs the extra black when x is red. When x is already the black root, remove the proof's extra contribution from every path without changing its stored color.

Left-side deficit: the full D1–D4 control flow

```
REPAIR-LEFT(T,x):
    p=x.parent; w=p.right
    if w.color==R:                          // D1
        w.color=B; p.color=R; LEFT-ROTATE(T,p)
        w=p.right
    if w.left.color==B and w.right.color==B:  // D2
        w.color=R
        return p
    if w.right.color==B:                    // D3
        w.left.color=B; w.color=R; RIGHT-ROTATE(T,w)
        w=p.right
    w.color=p.color                         // D4
    p.color=B; w.right.color=B; LEFT-ROTATE(T,p)
    return T.root
```


Right-side deficit: write the mirror explicitly

```
REPAIR-RIGHT(T,x):
    p=x.parent; w=p.left
    if w.color==R:                          // D1 mirrored
        w.color=B; p.color=R; RIGHT-ROTATE(T,p)
        w=p.left
    if w.left.color==B and w.right.color==B:  // D2 mirrored
        w.color=R
        return p
    if w.left.color==B:                     // D3 mirrored
        w.right.color=B; w.color=R; LEFT-ROTATE(T,w)
        w=p.left
    w.color=p.color                         // D4 mirrored
    p.color=B; w.left.color=B; RIGHT-ROTATE(T,p)
    return T.root
```

Select cases by sibling and near/far colors

Case	Condition, after handling a red sibling	Next state
D1	Red sibling	Black sibling; continue into D2/D3/D4
D2	Black sibling, both children black	Move the deficit to the parent
D3	Black sibling, red near/black far child	Convert to D4
D4	Black sibling, red far child	Remove extra black and terminate

For x on the left, near=w.left and far=w.right. Refresh w after D1 and D3. D3 falls through into D4. Returning p in D2 hands control back to the outer loop; it does not declare deletion complete.

For x on the right, near=w.right and far=w.left. Define these geometrically relative to x; near does not always mean left. Neither helper changes NIL.parent: the transplant establishes that context until repair finishes.

Shared NIL does not make the side test ambiguous here: the loop invariant requires a non-NIL sibling, so the parent cannot have NIL on both sides.

D1 converts a red sibling to a black sibling — black-count proof

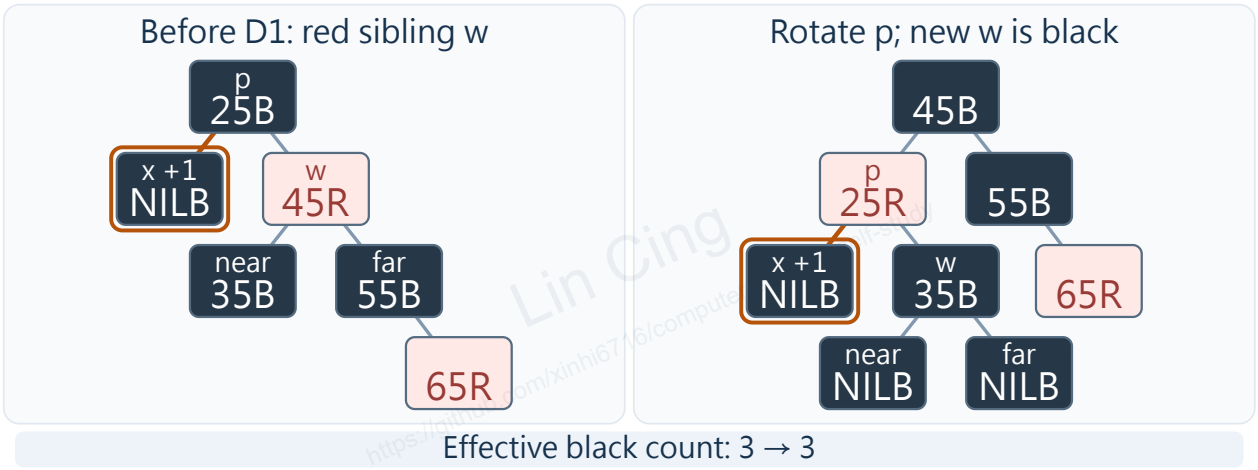
Let x's **actual** complete black count be b, including x, or NIL if x=NIL. Extra makes it b+1. Red w forces p and both w children black. Each w-child path contributes b+1, so every effective complete path including p contributes b+2.

Make w black and p red, then left-rotate p. New black local root w has a red left child p whose x-side effective count b+1 equals the old near-child count b+1. Its right side, the old far child, also contributes b+1. Total remains b+2. Extra remains at x; its new sibling is w's old black near child. Proceed to D2, D3, or D4.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

D1 converts a red sibling to a black sibling — worked trace

Insert 15,25,35,45,55,65 and delete 15. After removal x=NIL is 25B.left and w45R. D1 makes 45 black and 25 red, then left-rotates 25. New root 45B has left 25R(right 35B) and right 55B(right 65R); x remains 25.left. Complete black counts through ancestors are unchanged.



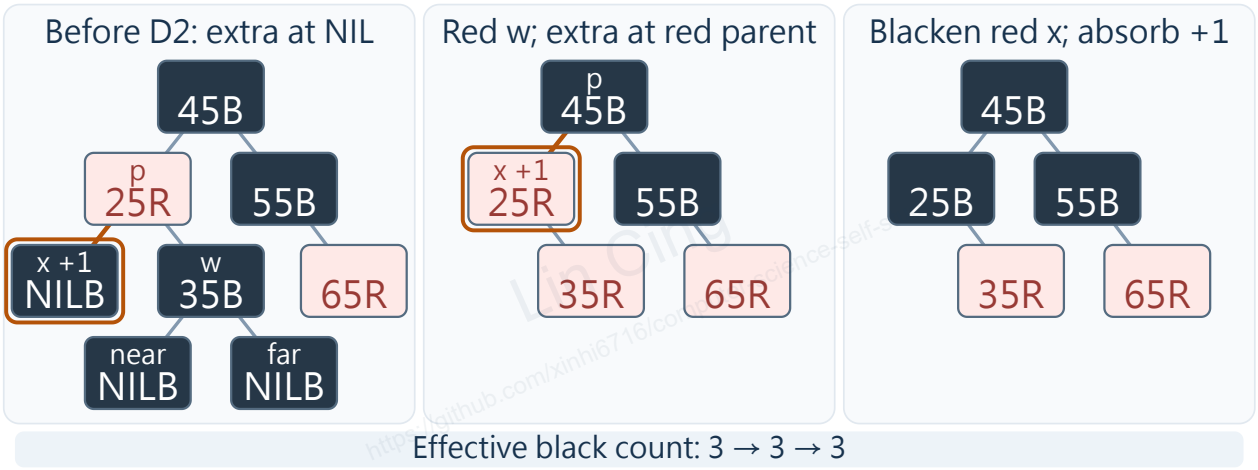
D2 moves the deficit upward or absorbs it at a red parent — black-count proof

Now w and both its children are black. x contributes actual b and effective b+1; w contributes b+1 and each child b. Making w red reduces its actual contribution to b. Both branches now have equal **actual** counts, but the whole p-subtree is one black short relative to its ancestors. Move extra to p by setting x=p.

If p was red, final blackening absorbs the extra and stops. If p was black and nonroot, iterate upward. At the root, drop extra consistently from all paths. D1 just made p red, so D1 followed by D2 absorbs in this round and cannot climb to another D1.

D2 moves the deficit upward or absorbs it at a red parent — worked trace

Continue the example: w35B has NIL children. Make 35 red and move x from NIL to 25R. The loop exits and 25 becomes black. Finally 45B has left 25B(right 35R) and right 55B(right 65R).



Effective black count: 3 → 3 → 3

D2 at the root reduces every path by one black

Start with 25B and black leaves 15B,35B. Deleting 15 leads D2 to make 35 red and move x to root 25. Dropping extra from all root-leaf paths leaves 25B(right 35R). Root bh decreases from 2 to 1, with equal path counts. This differs from absorption by blackening a red parent.



Effective black count: 3 → 3 → 2

D3 moves an available red node to the far side — black-count proof

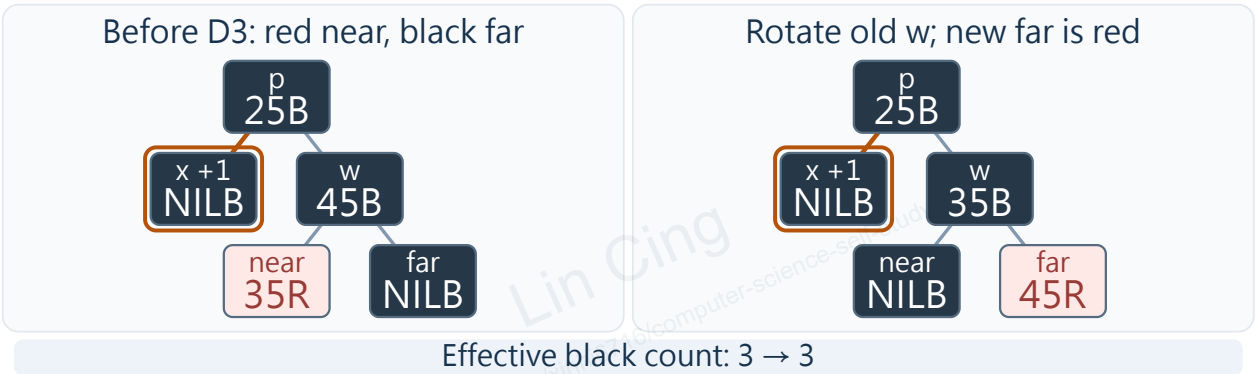
w is black, near N is red, and far is black. Let x's actual count be b. Both w children contribute b; red N's children each contribute b, as does far. Make N black and w red, then right-rotate w. New black sibling N has old N.left contributing b and old red w whose old N.right and far also contribute b. Its total remains b+1, preserving everything above p.

New sibling's far child is old red w, so execute D4 immediately. Extra remains at x; this case neither moves the deficit upward nor terminates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D3 moves an available red node to the far side — worked trace

Insert 15,25,45,35 to obtain 25B with left 15B and right 45B(left 35R). Delete 15, leaving x=NIL, w45B, near 35R, and black far=NIL. Make 35 black and 45 red, right-rotate 45, and obtain 25B with right 35B(right 45R). D4 must follow.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D4 repairs both sides by recoloring and rotation — black-count proof

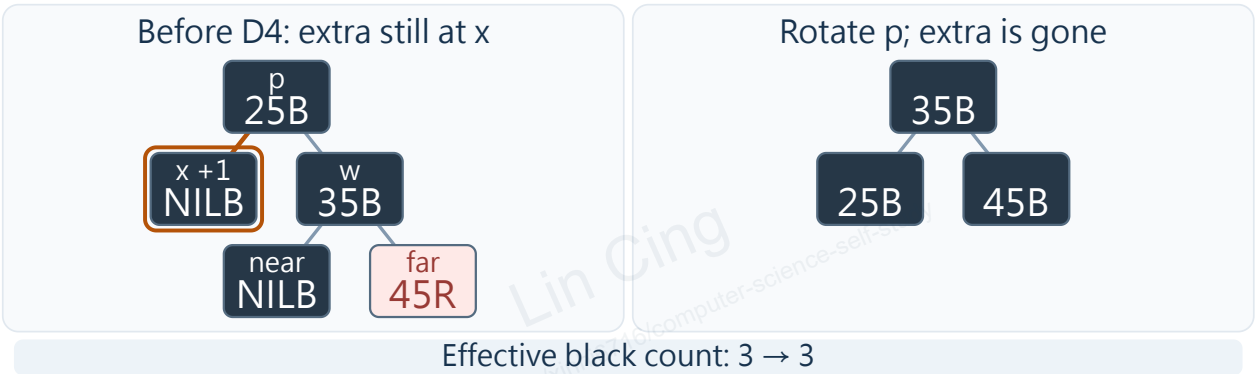
Let p's original black contribution be c∈{0,1}, x's actual count b, and both children of black w contribute b; far is red. Give w p's old color, make p and far black, and left-rotate p. New root w has black left child p with equal actual b on x and near, so its complete left count is b+1. Far changes red to black and also contributes b+1. New local total is c+b+1, equal to original p's effective c+(b+1).

If new w is red, its children p and far are black. Making p black creates no red-red conflict, and w retains p's external color relationship. Extra is eliminated. Set x=root to stop and ensure the root is black.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

D4: finish the numeric example

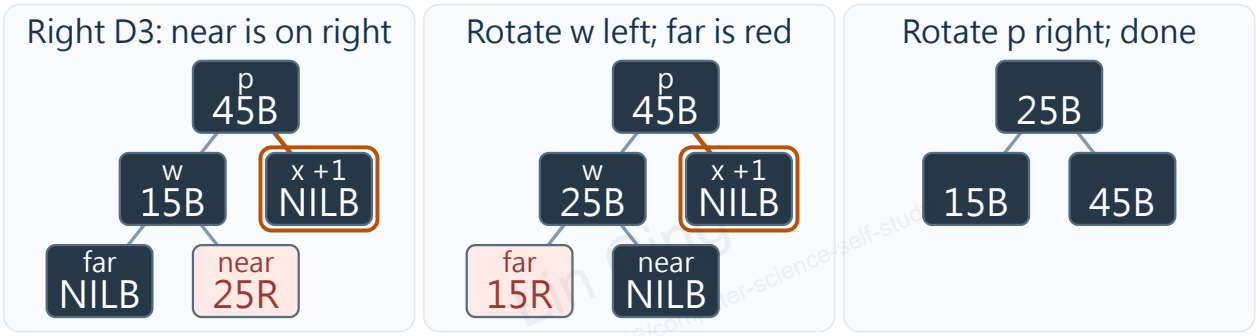
Continue D3:35 takes 25's old black color;25 and far 45 become black. Left-rotate 25 to obtain 35B with 25B,45B children. Inorder is 25,35,45, and every root-NIL path contains 3 blacks with no extra.



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Right-side D3 → D4

Mirror example: insert 15,35,45,25, then delete 35. Direct successor 45B becomes root; x=NIL is 45.right, so NIL.parent must be 45. Sibling 15B has near 25R and far=NIL. Right-side D3 left-rotates 15, then right-side D4 right-rotates 45, yielding 25B with 15B,45B children.



Effective black count: 3 → 3 → 3

Count black nodes in the earlier left-side D3→D4 example

Every row **includes its starting node and NIL**. D3 leaves x=NIL in place; D4 removes the extra black.

Stage and path	Actual black	Extra	Effective total
Before D3: 25B → NIL(x)	2	1	3
Before D3: 25B → 45B → 35R → NIL	3	0	3
After D3: 25B → 35B → 45R → NIL	3	0	3
After D4: 35B → 25B → NIL	3	0	3
After D4: 35B → 45B → NIL	3	0	3

Each black node adds 1 and each red node adds 0. Add the extra only on paths through x. The other NIL paths also total 3 by the same rule; check them in the figures. After D4 both sides have the needed black without an extra.

From local conservation to the general proof and cost

BST relocation preserves order and every undeleted record. Removing red directly leaves a valid tree. Removing black assigns extra to x so effective ancestor-to-leaf counts remain balanced. D1 and D3 preserve its position and effective counts; D2 moves the sole extra upward; D4 eliminates it. A red x is repaired by final blackening. In D2, newly red w has black children; rotations and recolorings satisfy the checked local color conditions. At termination all five properties hold.

D2 ascends one level with no rotation, at most O(log(n+2)) times. After D1, p is red, so D2 absorbs, or D3/D4 terminates. Therefore D1, D3, and D4 occur at most once each, for at most 3 rotations. Searching for z and its successor plus relocation and repair totals O(log(n+2)). NIL, root, and red-child absorption are valid base cases; deleting the last node leaves black root=NIL.

The rotation bound limits structural rearrangements. D2 propagation and recoloring may still traverse an ancestor path, so complete deletion takes O(log(n+2)), not O(1).

Dynamic order statistics from subtree sizes

Repeated updates and rank/select queries motivate an **order-statistic tree**: a balanced search tree augmented with subtree size, including the node itself. NIL.size=0 and

$$\text{size}(x) = 1 + \text{size}(x.left) + \text{size}(x.right).$$

Ranks are 1-based inorder positions of node identities. Examples use distinct keys; duplicates require a declared order such as (key,identity). A value alone does not identify one duplicate's rank.

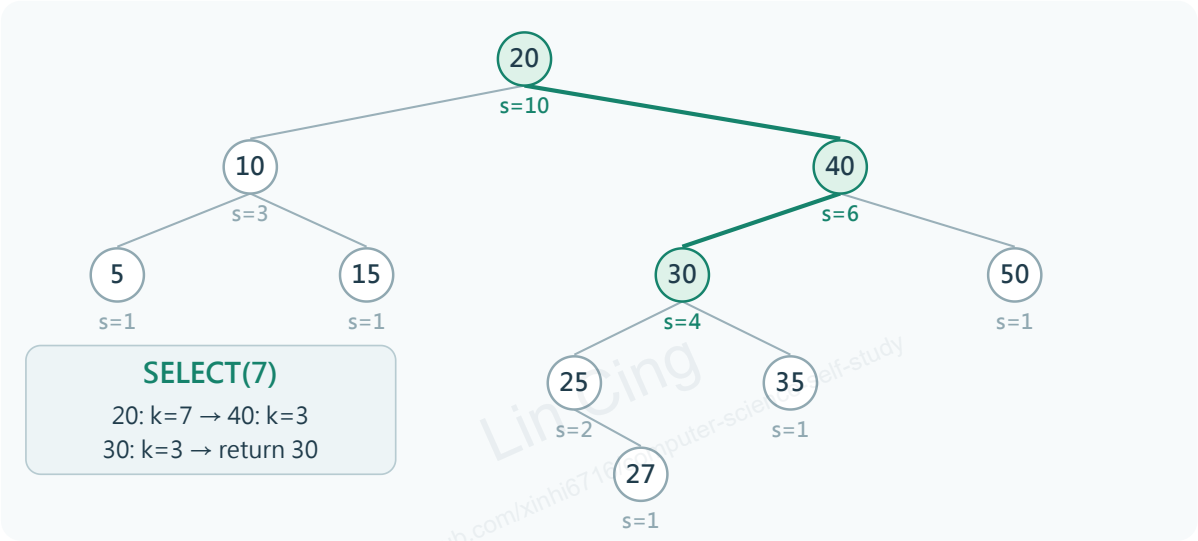
An augmentation must specify the summary, its maintenance after every modification, and the correctness of new queries. Here summaries use local data and child summaries in O(1) and do not depend on colors; color-dependent summaries require separate recoloring costs. First bound costs by height h: O(h+1). Red-black invariants then give O(log(n+2)); adding a field does not balance an ordinary BST.

Select uses a local rank at every level

```
select(x,k):
    require 1<=k<=x.size
    while x!=NIL:
        r=x.left.size+1
        if k==r: return x
        if k<r: x=x.left
        else: k-=r; x=x.right
```

Invariant: the answer is local rank k in the current x-subtree. The left subtree has r−1 nodes and x is position r. Going left retains k; going right subtracts the entire left subtree plus x, r nodes. A valid k guarantees the chosen side is nonempty and its new k valid, so descent never incorrectly enters NIL. Each iteration descends one level and returns the correct node.

One tree: keys, subtree sizes, and the selection path



Keys appear inside circles; s below each circle is subtree size. Green traces SELECT(7): 20→40→30. Rank and deletion examples start from the same tree.

Select(7): subtract only the skipped prefix

Current key	k	left.size	r=left.size+1	Action and new k
20	7	3	4	Right to 40; 7−4=3
40	3	4	5	Left to 30; still 3
30	3	2	3	Return this node

Inorder is 5,10,15,20,25,27,30,35,40,50, confirming key 30 at rank 7. **Subtract the left subtree and current node only when moving right.** Reject k=0 or k>root.size before querying. An empty tree has no valid k.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rank adds disjoint prefixes while climbing

```
rank(T,x):
    r=x.left.size+1; u=x
    while u!=T.root:
        if u==u.parent.right:
            r+=u.parent.left.size+1
        u=u.parent
    return r
```

Invariant: r is original x's inorder rank inside current u's subtree. If u is a left child, the parent and right side follow x, so add nothing. If u is a right child, its parent's left subtree and parent precede x and lie outside the previously counted u-subtree. Add exactly left.size+1 without duplication. At root, r is the global rank.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rank(27): which ancestors add a prefix?

Find the rank of node 27, starting with r=0+1=1.

u → parent	u's side	Added prefix	New r
27 → 25	right	size(NIL)+1=1	2
25 → 30	left	0	2
30 → 40	left	0	2
40 → 20	right	size(10)+1=4	6

Each added set lies outside the old u subtree and precedes 27. The sets are disjoint and exhaustive, confirming inorder rank 6. This API requires a valid node belonging to the tree; NIL and removed objects are invalid arguments.

Repair summaries bottom-up, including during rotations

```
PULL(v):                      # v != NIL
    v.size=1+v.left.size+v.right.size
    v.max=max(v.high,v.left.max,v.right.max)
REPAIR-UP(v):
    while v!=NIL:
        PULL(v); v=v.parent
```

This version stores both size and the largest interval high; omit max in a size-only tree. NIL.size=0 and NIL.max=−∞. A new leaf has size 1; after attaching it, repair upward from its parent.

For deletion, finish BST relocation, repair along final parent links, and then run red-black fixup. After changing rotation pointers, pull the lowered node before the promoted node. Recoloring does not affect size/max. Deletion or a shorter interval may decrease max; keeping a historical upper bound is unsafe.

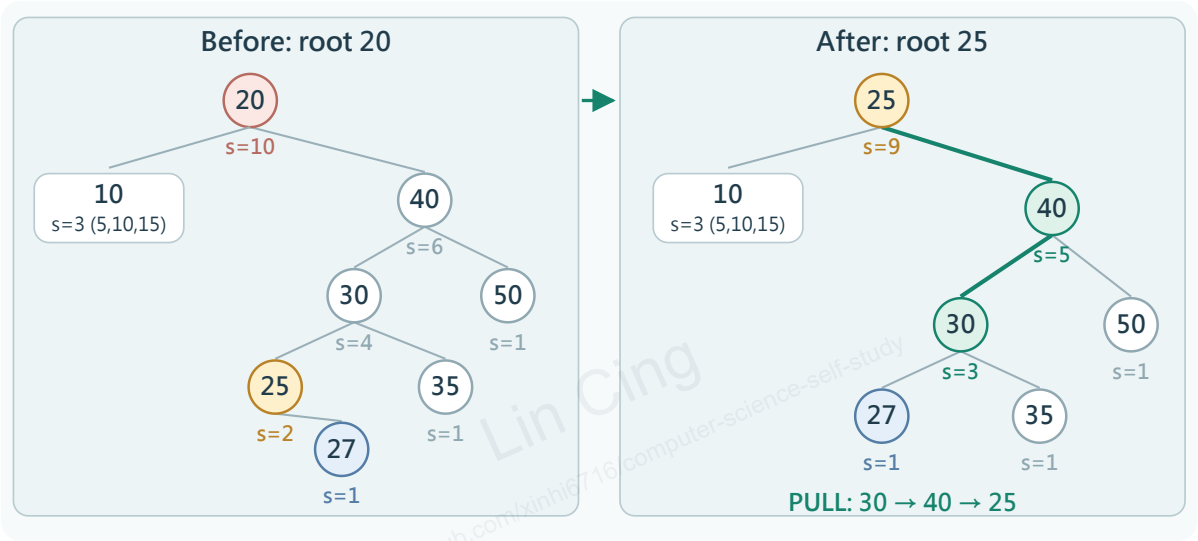
Where does summary repair start after deletion?

In the preceding RB-DELETE/RELOCATE-SUCCESSOR, save a before relocation according to the table. After all transplants, run REPAIR-UP(a), then repair colors.

Branch	Saved a	Why this final chain suffices
z has at most one child	z's old parent	The child's own subtree is unchanged; ancestors lose one record
Successor y is z.right	y	y has a new left subtree; repair upward from y
Successor y is deeper	y's old parent	Its final chain visits the old right subtree, moved y, and z's old ancestors

Never read z.parent after freeing z. NIL.parent is not a universal parent for all empty links. Save a separately as a real node or NIL; NIL.size=0 and NIL.max=−∞ never change.

Delete 20 and relocate 25: track links and sizes



Node 25 keeps its identity;27 replaces it under 30, and 25 takes the old root position. Colors distinguish transplant roles, not red-black colors. Complete BST relocation and summary repair before the preceding red-black fixup.

Pull 30, then 40, then the new root 25

Successor 25 has right child 27. Replace 30.left by 27, then move 25 to the root with old left subtree 10 and right subtree 40.

Repair order	Calculation	New size
30	1+size(27)+size(35)=1+1+1	3
40	1+size(30)+size(50)=1+3+1	5
25	1+size(10)+size(40)=1+3+5	9

The old size of 25, namely 2, is obsolete. Pulling 25 first reads stale 40.size=6 and wrongly produces 10. Merely decrementing z's old ancestors updates nothing here because z was the root.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

When can a rotation update only two summaries?

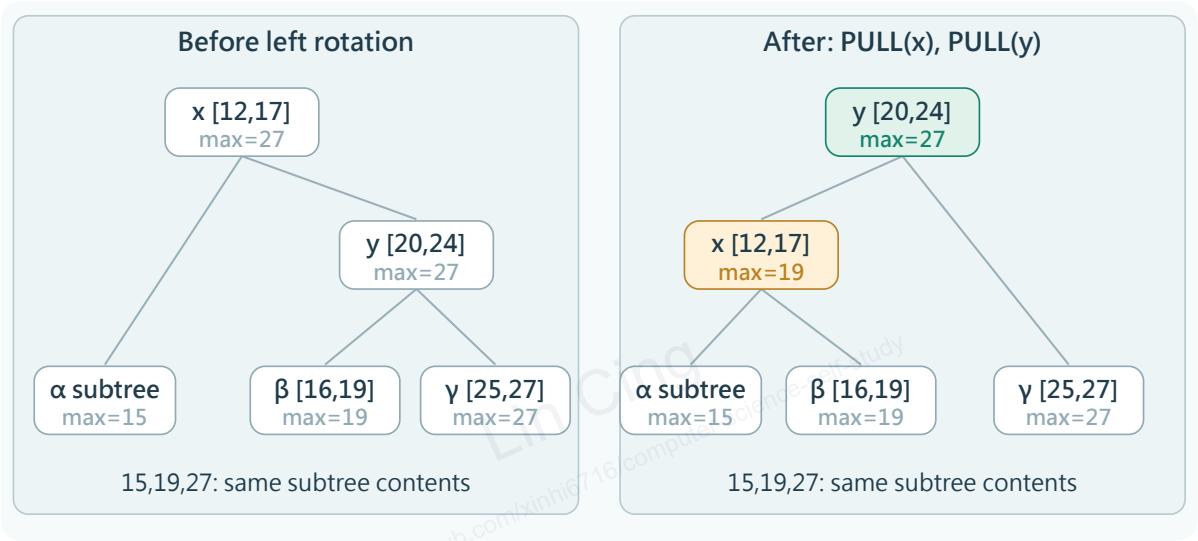
In left-rotate(x), y=x.right rises and β=y.left moves to x. Finish pointer changes, then pull(x) before pull(y), because new y depends on new x. Reversing the order reads stale x information.

Size and maximum interval high depend only on subtree contents. New y contains exactly old x's entire subtree after rotation, so its summary equals old x's summary and other ancestors do not change. These two summaries need O(1) local recomputation.

A local recurrence alone does not imply a two-node update. Height can change at the local root after rotation, requiring ancestor propagation. A general constant-time child-based summary costs O(h+1) per rotation; the constant rotation bounds of red-black updates retain O(log(n+2)) overall. Storing global ranks instead would require Θ(n) explicit writes when a new minimum is inserted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The same interval set before and after rotation



First x.max=max(17,15,19)=19; then y.max=max(24,19,27)=27. The new local root still represents the entire old set.

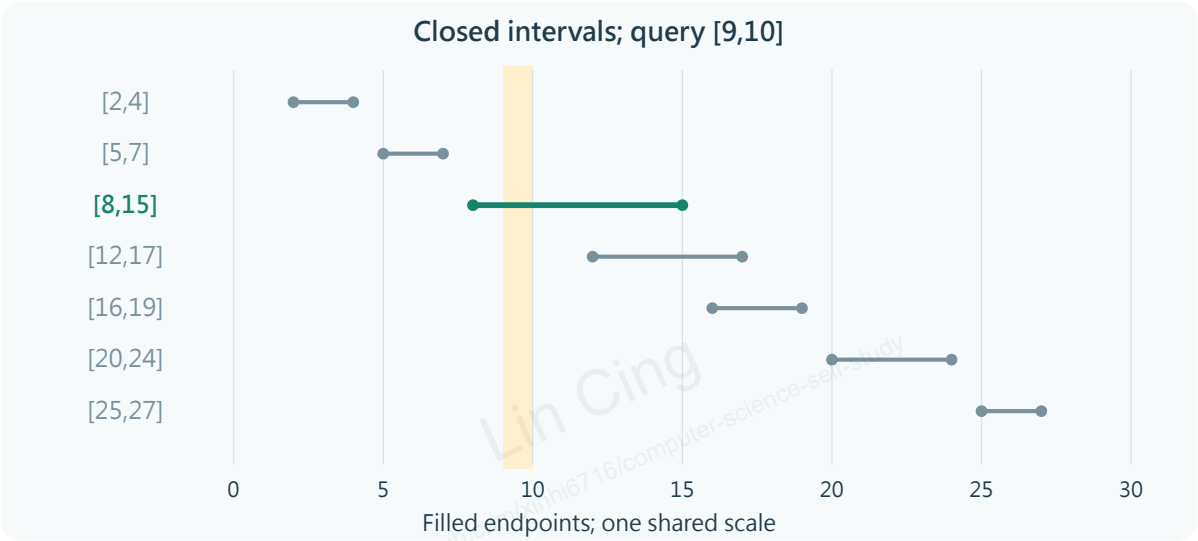
Interval trees: order by low, summarize high

Intervals here are closed [l,h], with l≤h, including singletons [l,l]. Intervals i,q overlap exactly when i.low≤q.high and q.low≤i.high. Equivalently, neither lies completely left of the other: i.high<q.low and q.high<i.low are the two rejection cases.

Order the BST primarily by low; use identity to break equal-low ties, retaining left.low≤x.low≤right.low. Store max(x)=max(x.high,left.max,right.max), with NIL.max=−∞. This is the highest high in the **whole subtree**, not the node's high or the largest low.

Root [12,17] has high 17 but max 27. Left[5,7] has max 15 from right descendant [8,15]. Looking only at the current interval can miss a farther-reaching descendant.

All seven intervals share one coordinate scale



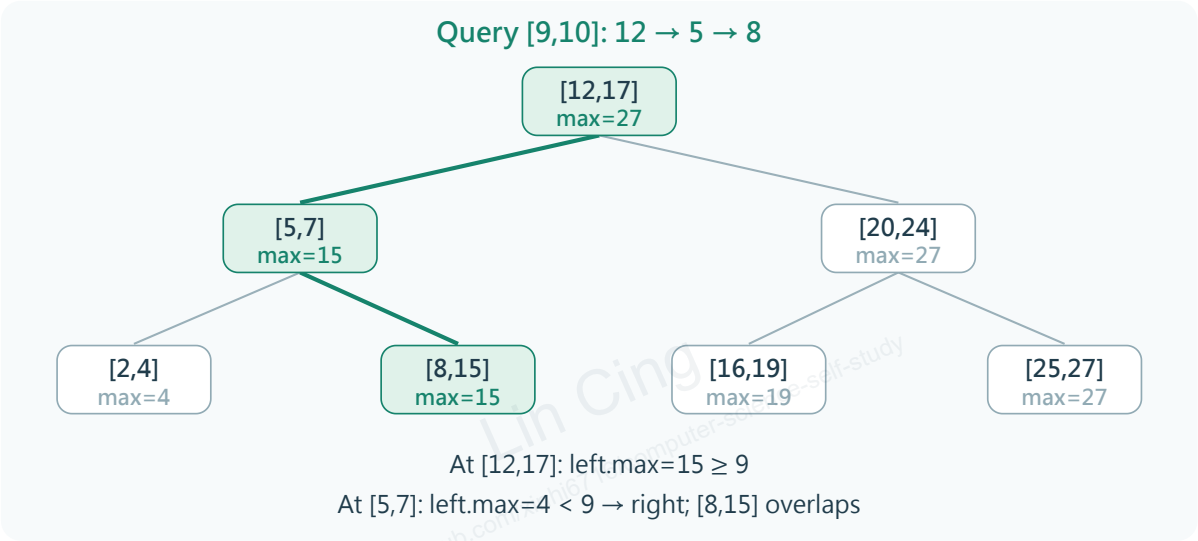
The shaded query is [9,10]; only [8,15] overlaps it. The tree uses exactly these intervals.

Overlap search chooses one subtree at a time

```
interval-search(T,q):
    x=T.root
    while x!=NIL and not overlaps(x.interval,q):
        if x.left!=NIL and x.left.max>=q.low:
            x=x.left
        else:
            x=x.right
    return x
```

This returns any one overlap, not the leftmost or all matches. Prove that every chosen subtree retains an answer whenever the current subtree has one.

One interval tree and the query [9,10] path



Read left.max at the current node: 15≥9 chooses left; then 4<9 chooses right and reaches [8,15].

Three queries with traceable branch conditions

Query [9,10] does not overlap root [12,17], but left.max 15≥9 leads to [5,7]. That also misses; left [2,4].max 4<9 leads right to overlapping [8,15].

Query [18,18] goes from nonoverlapping root right because left.max 15<18. At [20,24], left.max 19≥18 leads to matching [16,19]. Query [28,29] follows [12,17]→[20,24]→[25,27]→NIL and fails.

Going right: rule out the entire left subtree

Maintain: if an overlap exists anywhere in the tree, the current x-subtree contains one. This holds initially at root. Return if x overlaps. Otherwise prove that the chosen direction cannot discard all answers.

If left=NIL, it has no candidate. If left.max<q.low, every left interval j has j.high≤left.max<q.low and lies completely left of the query. Exclude the whole left side. Since x also misses, any answer must be right, preserving the invariant.

This uses a bound on every left descendant's high. The single condition x.high<q.low does not imply every left interval lies too far left. Change only the example root to [12,13] and query [14,14]: root.high 13<14, yet left descendant [8,15] overlaps, so the left side cannot be pruned on that basis.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Going left: why no right-side backtracking is needed

When left.max≥q.low, choose a left interval j attaining that maximum. If left has any overlap, descending left is safe. If it has none, j.high≥q.low while j misses, so j.low>q.high; it cannot have j.high<q.low.

For every right descendant k, BST order gives `k.low≥x.low≥j.low>q.high`. The whole right side also lies too far right. With x itself nonoverlapping, this means the x-subtree had no answer. Thus either way, if an answer existed, the chosen left side retains one.

This proves the one-path algorithm. It does not assert that left.max≥q.low guarantees a left overlap. At NIL the chosen subtree is empty, so the invariant implies global absence. Each step descends once, giving O(height+1), or O(log(n+2)) with a red-black foundation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

An overestimated stale max can miss an answer

Change [8,15] to [8,27], then delete it. The left subtree now contains only [5,7] and [2,4], so its exact max is 7. If it incorrectly remains 27, query [22,22] behaves as follows.

Position	Comparison used	Incorrect choice
[12,17]	Stale left.max=27≥22	Left, abandoning the real answer on the right
[5,7]	left.max=4<22	Into empty right child
NIL	Loop ends	Incorrect failure

The actual answer is [20,24] in the right subtree. With exact left.max=7 the first step goes right and finds it. **This one-path algorithm requires an exact maximum, not just an upper bound.** Underestimates can prune true answers directly.

Recompute max bottom-up after modifications, allowing decreases.

Endpoint and update conventions are part of the contract

Convention	Nonempty condition	overlap(i,q)	Left may contain an answer
Closed [l,h]	l≤h	i.low≤q.high and q.low≤i.high	left.max≥q.low
Half-open [l,h)	l<h	i.low<q.high and q.low<i.high	left.max>q.low

[4,5] and [5,7] meet at 5; [4,5) and [5,7) do not overlap. Half-open [5,5) is empty, unlike a singleton closed interval.

Changing low changes the BST ordering key: delete and reinsert the record instead of editing the field in place. Changing only high preserves order, provided the interval stays nonempty, but requires max repair upward from that node.

One match, leftmost match, and all matches differ

The algorithm promises any overlapping record, not minimum low or all matches. To enumerate all, recurse left only if left.max≥q.low, test x, and recurse right only if x.low≤q.high and right.max≥q.low. Each visited node costs O(1), with worst-case O(n). The one-path O(log(n+2)) bound cannot be transferred to enumeration.

Another provable enumeration method repeats overlap-search, temporarily deletes each returned record, and records it in a list. k matches plus one final failure cost O((k+1)log(n+2)); reinsertion has the same order. This temporarily mutates the tree and must fit the caller's contract. Output alone costs Ω(k). Achieving exactly O(log n+k) requires another analysis or representation; saying “output-sensitive” is no proof.

Size and max can be stored together, each pulled in O(1), with the same maintenance order. Rank then follows BST order on (low,identity), not high order or number of overlaps, which need other summaries or algorithms.

B-trees: use a page to reduce tree height

A node stores several sorted keys and child pointers separating their ranges. Search chooses a range within the current page, then reads another page. Separate **page accesses (I/O)** from **within-page comparisons and movement (CPU)**.

We use a set of distinct keys. Each key travels with its payload or record pointer; internal nodes can store actual records. Duplicate insertion returns EXISTS without adding a record. A B+ tree uses a different representation: records reside in leaves and internal nodes store separators.

Whether one node fits a page depends on page size, key/payload sizes, child pointers, and headers. The I/O analysis assumes this layout requirement holds.

Minimum degree t: fix the occupancy contract

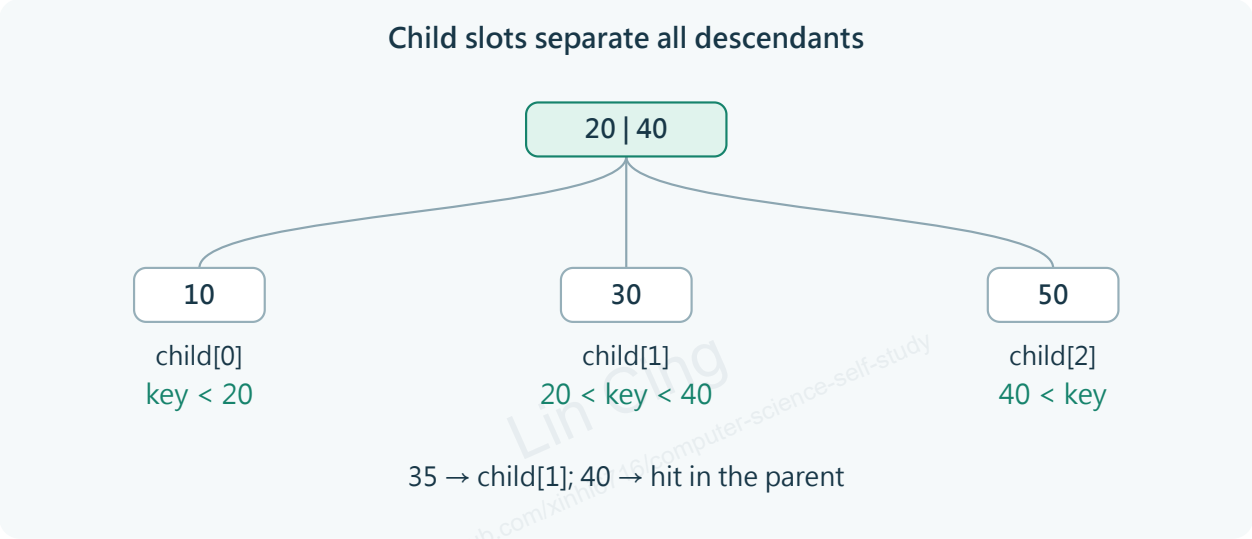
Fix an integer $t \geq 2$. Key and child arrays are zero-based.

Node type	Key count k	Child count
Nonroot	$t-1 \leq k \leq 2t-1$	0 for a leaf; exactly k+1 otherwise
Nonempty root	$1 \leq k \leq 2t-1$	0 for a leaf; at least 2 otherwise
Empty tree	Zero-key leaf root	0

All leaves have the same depth. The root has depth 0 and leaf depth is height h. A nonempty height-zero tree stores everything in its root. We also assign height 0 to the empty tree, but the nonempty height theorem does not apply at n=0.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Every child pointer owns a whole key range



Keys $a_0 < \ldots < a_{k-1}$ divide the subtree into k+1 ranges. Each restriction applies to **every descendant**, not merely the first key of a child. Search for 35 chooses the middle child; search for 40 stops at the parent.

Do not mix order m with minimum degree t

Convention used here	Nonroot internal children	Nonroot keys
Order m means at most m children	$\lceil m/2 \rceil \ldots m$	$\lceil m/2 \rceil - 1 \ldots m-1$
Minimum degree t	$t \ldots 2t$	$t-1 \ldots 2t-1$

Capacities match exactly when m=2t. Order 3 gives a 2-3 tree. It is neither t=1.5, which is invalid, nor t=2, which permits four children and gives a 2-3-4 tree.

Books do not use “order” uniformly. Check whether it means maximum children or another capacity before applying a formula. The table explicitly fixes the maximum-child convention.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Preemptive and overflow splitting have different premises

Our insertion splits a full node with $2t-1$ keys before descending: promote one key, leaving $t-1$ keys on each side. Both sides are immediately legal.

An order-m overflow algorithm first permits m temporary keys, then promotes a median. With m=3, inserting 5 into [2,8] temporarily gives [2,5,8]; promote 5 and retain [2] and [8].

Preemptively splitting [2,8] at order 3 leaves one side empty regardless of which key rises, violating the one-key minimum. **Capacity, repair timing, and proof must agree.** Deletion likewise has both repair-before-descent and repair-after-underflow schedules.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Height upper bound: count the sparsest legal tree

Assume $n \geq 1$. The root has at least one key and, if $h \geq 1$, at least two children. Depth $i \geq 1$ contains at least $2t^{i-1}$ nodes, each with at least $t-1$ keys. Thus

$$n \geq 1 + \sum_{i=1}^{h} 2t^{i-1}(t-1) = 2t^h - 1.$$

Rearranging and taking base-t logarithms gives

$$h \leq \log_t \frac{n+1}{2}.$$

For $h=0$ the sum is empty, giving $n \geq 1$ as required. The root minimum is one key; do not count it as another node with $t-1$ keys.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Height lower bound and a billion-key example

Each node has at most $2t-1$ keys and $2t$ children, so

$$n \leq (2t-1)\sum_{i=0}^{h}(2t)^i = (2t)^{h+1} - 1.$$

For $t=100$ and $n=10^9$, height 2 holds at most 7,999,999 keys, too few; height 3 holds at most 1,599,999,999. Height 4 requires at least 199,999,999 keys, while height 5 requires 19,999,999,999, too many. A legal tree therefore has $h=3$ or 4.

Search visits 4 or 5 nodes. With a resident root and no other path pages cached, it performs **3 or 4 page reads**. A nonresident root adds one more read.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Search: locate a slot, then choose the only possible subtree

```
SEARCH(x,k):
    i=lower_bound(x.keys,k)       // first key >= k, or len(keys)
    if i<len(x.keys) and x.keys[i]==k: return (x,i)
    if x.children is empty: return NOT_FOUND
    read x.children[i] if not cached
    return SEARCH(x.children[i],k)
```

Without an equal key, global range separation leaves only child i as a possible location. Every step descends one level and eventually succeeds or fails at a leaf. This proves termination and correctness. Linear scanning and binary search implement the same slot choice and page path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

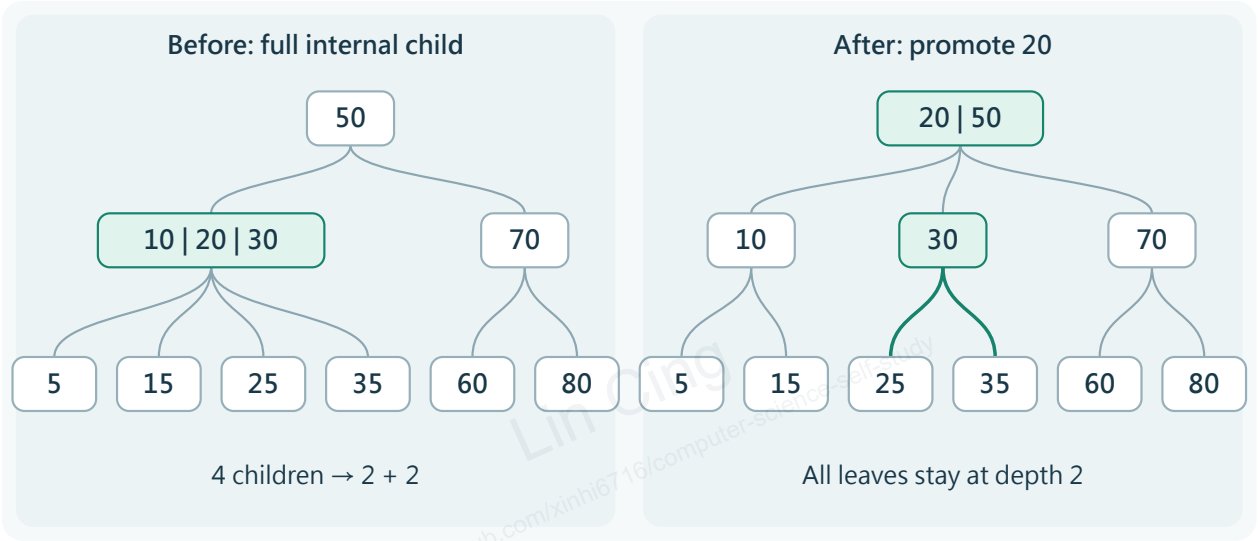
SPLIT: move the median and divide the child pointers

```
SPLIT(p,i):                       // p nonfull; child i full
    y=p.children[i]; old=y.keys; kids=y.children
    median=old[t-1]
    z=Node(keys=old[t:], children=kids[t:] if kids else [])
    y.keys=old[:t-1]
    y.children=kids[:t] if kids else []
    p.keys.insert(i,median); p.children.insert(i+1,z)
    write y,z,p
```

Slices include the left endpoint and exclude the right. old and kids refer to the pre-change arrays. Both halves have t−1 keys and, if internal, t children. The median moves to the parent, with **no copy retained in either child**. A nonfull parent can safely gain one key.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splitting an internal node must preserve all four child subtrees

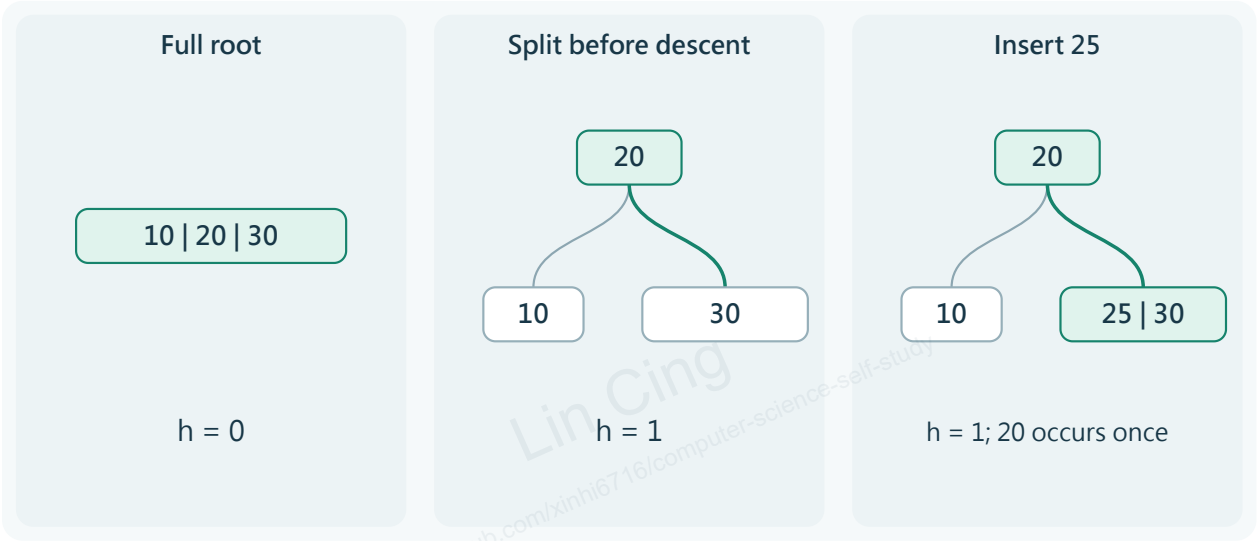


For t=2, parent [50] has full left child [10,20,30]. Promoting 20 gives parent [20,50]. Node [10] keeps [5],[15]; node [30] receives [25],[35]. Old right subtree [70] is unchanged. All leaves remain at depth 2.

Insertion: always descend into a nonfull node

```
INSERT(k):
    if SEARCH(root,k) succeeds: return EXISTS
    if len(root.keys)==2*t-1:
        root=Node([], [root]); SPLIT(root,0)
    x=root
    while x.children is not empty:
        i=lower_bound(x.keys,k)
        if len(x.children[i].keys)==2*t-1:
            SPLIT(x,i)
            if k>x.keys[i]: i+=1
        x=x.children[i]
    insert k in sorted order into x.keys; write x
    return INSERTED
```

Root splitting increases height at the top



For t=2, insert 25 into full leaf root [10,20,30]. Promote 20 first, then choose the right leaf since 25>20, yielding [25,30]. Only root splitting adds a level; all old leaves deepen together and remain at equal depth.

Insertion correctness: why no upward repair is needed

At loop entry, x is nonfull. A full original root is split under a new root, establishing the invariant initially. If the next child is full, nonfull x can accept its promoted key. Both split children have t−1 keys, so the chosen child is again nonfull.

Each iteration descends one level. A leaf gains just one key, at most from 2t−2 to 2t−1, without overflowing. SPLIT preserves keys and subtree ranges; the final new key lies in the range chosen by search. The represented set gains exactly k.

Repair precedes descent, unlike an overflow-based schedule, so insertion need not return to an earlier parent to fix overflow. The separate duplicate check adds another search path but does not change the asymptotic bound.

Deletion: give the next subproblem one key of slack

Ordinary nonroot occupancy is $t-1$ keys. Deletion strengthens the entry condition: **before recursing into a nonroot node, ensure it has at least t keys**. Losing one key at that level then leaves the legal $t-1$ minimum.

Situation	Action
k found in a leaf	Remove it
k found internally; left child has $\geq t$ keys	Replace by predecessor; delete it on the left
Left child lacks t; right has $\geq t$	Replace by successor; delete it on the right
Both adjacent children have $t-1$	Merge both with the parent key, then delete k

Even when k is not found locally, repair its target child before descending. The root is exempt from the stronger minimum; a final zero-key internal root is replaced by its only child.

Borrow left: lower the separator and raise the sibling maximum

```
BORROW-LEFT(p,i):                // i>0
    y=p.children[i]; s=p.children[i-1]
    y.keys.insert(0,p.keys[i-1])
    p.keys[i-1]=s.keys.pop()
    if s.children: y.children.insert(0,s.children.pop())
    write s,y,p
```

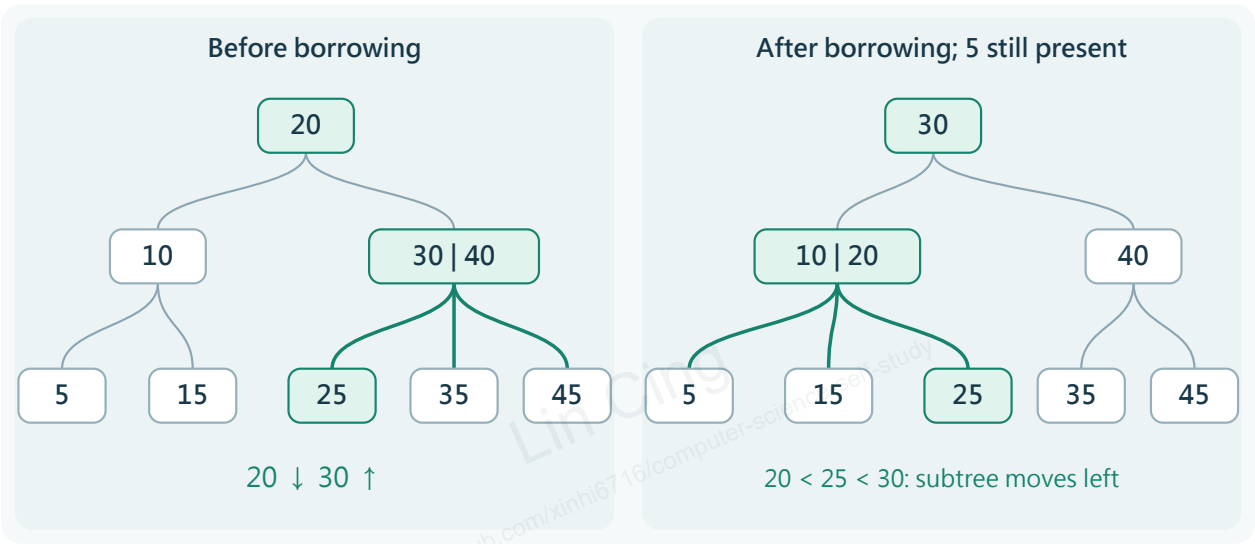
Require $t-1$ keys in y and at least t in s, at the same depth. The rightmost subtree of s lies between the new and old separators and must become the leftmost subtree of y. Afterward y has t keys, s retains at least $t-1$, and the parent key count is unchanged.

Borrow right: move the child pointer to the correct side

```
BORROW-RIGHT(p,i):                // i<len(p.keys)
    y=p.children[i]; s=p.children[i+1]
    y.keys.append(p.keys[i])
    p.keys[i]=s.keys.pop(0)
    if s.children: y.children.append(s.children.pop(0))
    write s,y,p
```

Again require t−1 keys in y and at least t in s. The leftmost child of s becomes the rightmost child of y. Moving keys without moving that pointer assigns an entire key range to the wrong subtree, even if every individual node remains sorted.

Borrowing example: why the 25 subtree must move left



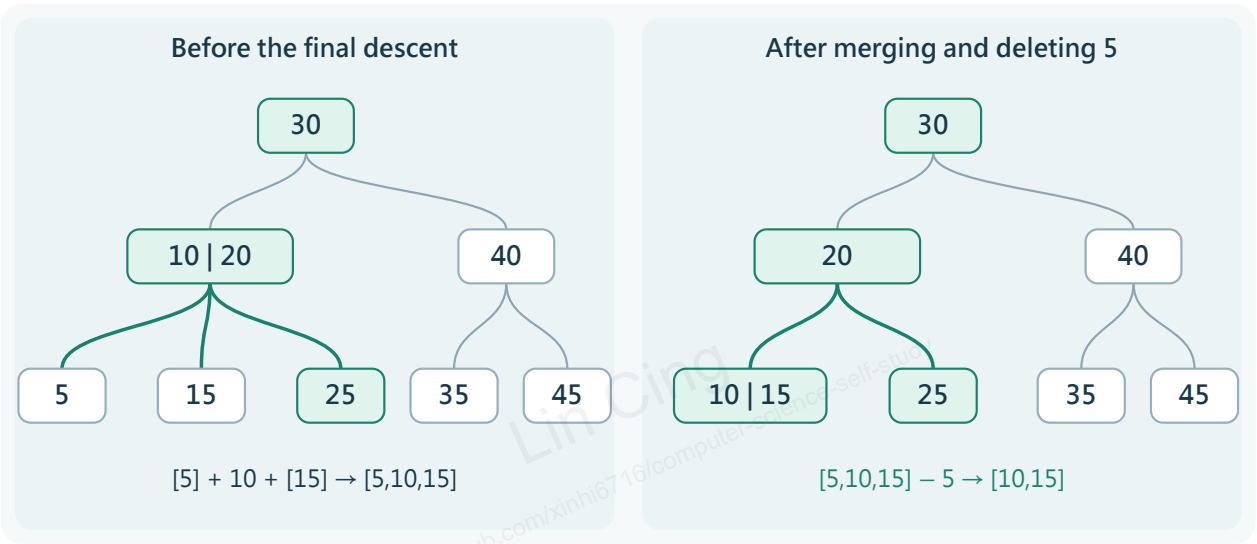
For t=2, delete 5. Left internal node [10] has only one key; right sibling [30,40] has two. First borrow: 20 descends and 30 rises. Subtree [25] lies between 20 and 30 and must become the last child of [10,20]. The figure shows pre-descent repair; 5 has not yet been deleted.

MERGE includes the parent separator

```
MERGE(p,i):                     // adjacent children both t-1 keys
    y=p.children[i]; z=p.children[i+1]
    y.keys.append(p.keys.pop(i))
    y.keys.extend(z.keys)
    y.children.extend(z.children)
    p.children.pop(i+1)
    write y,p; free z
    return y
```

New y has (t−1)+1+(t−1)=2t−1 keys: full but not overflowing. If internal, it has t+t=2t children. The parent loses one key and one child. A nonroot parent entered with at least t keys and retains t−1. An empty root is contracted after the operation.

Continue deleting 5: merge two minimal leaves first



Inside [10,20], target leaf [5] and sibling [15] each have one key. Include parent separator 10 to form [5,10,15], then remove 5, leaving [10,15]. The left internal node is [20] with children [10,15],[25]. Root [30] and right node [40] are unchanged. Inorder loses exactly 5 and all leaves remain at depth 2.

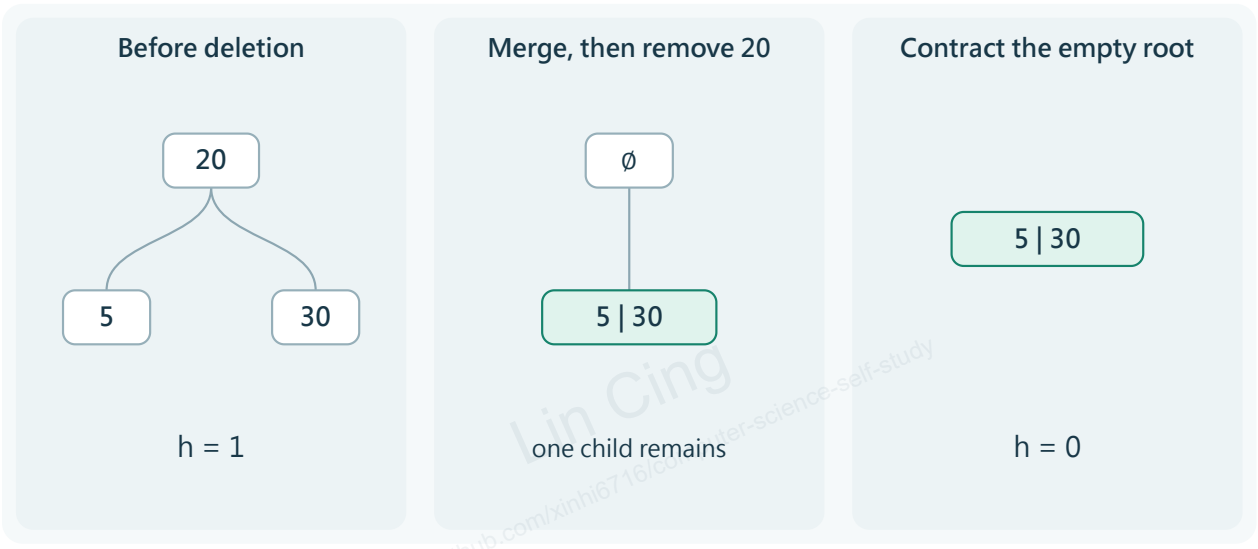
An internal match moves a whole predecessor or successor record

Initial t=2 tree: root and two leaves	Delete	Result
Root [20]; left [5,10], right [30]	20	Predecessor 10 rises: root[10], leaves[5],[30]
Root [20]; left [5], right [30,40]	20	Successor 30 rises: root[30], leaves[5],[40]

Find a predecessor by following the left subtree's rightmost children to a leaf and taking its last key. A successor follows the right subtree's leftmost children and takes the first key. Delete that record from its original subtree; do not retain duplicate parent/leaf copies.

The record pointer must move with its key. Changing only the key while leaving the old payload makes later queries return the wrong record.

Root contraction: only the root may remove a level



Merge root separator [20] with leaves [5],[30] into [5,20,30], then delete 20, leaving [5,30]. The old root has zero keys and one child; replace it by that child, reducing h from 1 to 0. Doing this to an ordinary internal node would shorten only one branch and violate equal leaf depth.

Deletion procedure: the three internal-match cases

```
DELETE-NODE(x,k):
    i=lower_bound(x.keys,k)
    if i<len(x.keys) and x.keys[i]==k:
        if not x.children: erase x.keys[i]; write x; return FOUND
        L=x.children[i]; R=x.children[i+1]
        if len(L.keys)>=t:
            q=MAX-RECORD(L); DELETE-NODE(L,q.key); x.keys[i]=q
        elif len(R.keys)>=t:
            q=MIN-RECORD(R); DELETE-NODE(R,q.key); x.keys[i]=q
        else:
            y=MERGE(x,i); DELETE-NODE(y,k)
        write x; return FOUND
    return DELETE-DESCEND(x,i,k)  // next card
```

Here the keys array contains records; comparisons use their key fields. q must be a saved record value or stable pointer, not an object dereferenced after its original storage is freed.

Deletion descent: borrow first; adjust the index after merging

```
DELETE-DESCEND(x,i,k):
    if not x.children: return NOT_FOUND
    if len(x.children[i].keys)==t-1:
        if i>0 and len(x.children[i-1].keys)>=t:
            BORROW-LEFT(x,i)
        elif i<len(x.keys) and len(x.children[i+1].keys)>=t:
            BORROW-RIGHT(x,i)
        elif i<len(x.keys): MERGE(x,i)
        else: MERGE(x,i-1); i-=1
    return DELETE-NODE(x.children[i],k)
```

Merging left absorbs old child i into child i−1, so decrement i. Public DELETE(k) calls DELETE-NODE(root,k), then **contracts a zero-key internal root on either FOUND or NOT_FOUND**, and returns the original search result.

An absent-key deletion can still change tree shape

Let t=2, root [20], and leaves [5],[30]. Delete 99: choose the right child. It has only t−1=1 key and its sibling cannot lend, so merge into [5,20,30] and change the child index from 1 to 0.

The resulting leaf lacks 99 and returns NOT_FOUND. The wrapper must still contract the empty internal root. The final tree is leaf root [5,20,30]: the set is unchanged, while height falls from 1 to 0.

NOT_FOUND promises no element removal, not an unchanged shape. An API requiring shape preservation on failure can first search for existence and account for that extra path. Empty trees and deletion of the last key retain a zero-key leaf root.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

General deletion proof and two separate cost measures

Induct on remaining height. Leaf removal is correct. Internal replacement deletes the predecessor or successor from its old subtree, removing exactly k overall. Borrowing and merging preserve global order, child counts, and equal leaf depth, while giving the next subproblem at least t keys. A nonroot parent losing a separator retains t−1. Root contraction handles the exception. Every recursive call descends, proving termination.

Only constantly many neighboring pages are touched per level: I/O is $O(h+1)=O(1+\log_t(n+1))$. Array movement costs up to O(t) per level, giving O(t(h+1)) insertion/deletion CPU time. Separate predecessor/successor lookup adds paths totaling O(h+1), not a fresh full-height traversal at every level.

Binary search inside a page gives O((h+1)log t) search comparisons. It does not reduce insertion/deletion array movement to O(log t).

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Fibonacci heaps: defer linking until extract-min

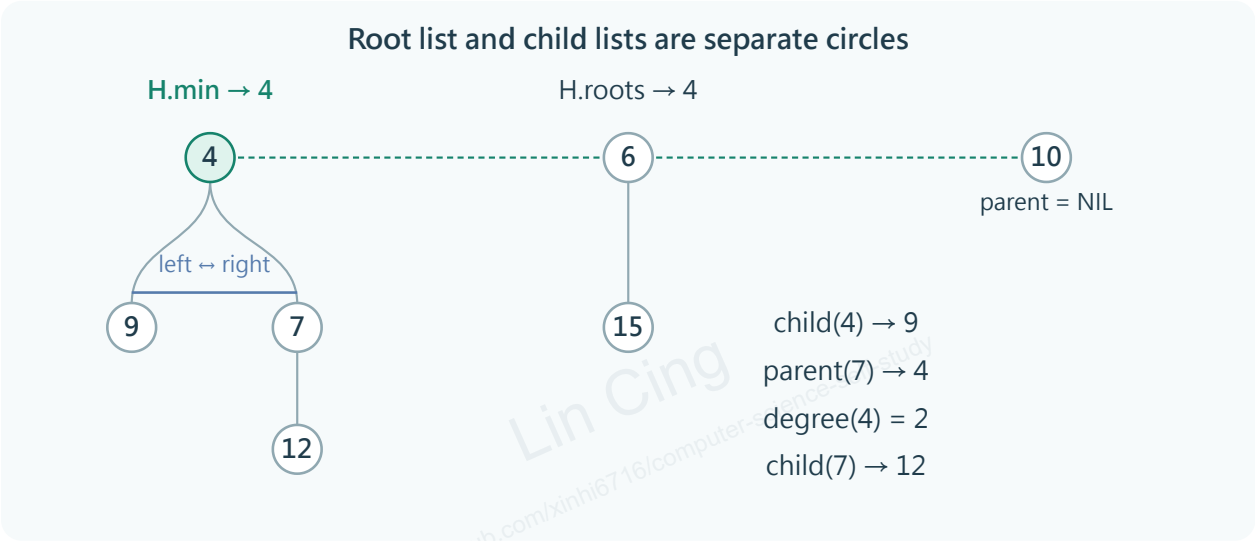
A min-heap requires each parent key to be no greater than its child keys; different subtrees need not be ordered. A Fibonacci heap keeps a list of such trees. Insertion and meld concatenate roots; EXTRACT-MIN later links roots of equal degree.

Degree is the **number of immediate children**, not height. Cuts destroy the exact binomial-tree shape. We will prove degree O(log n); this does not assert logarithmic root-to-leaf paths.

DECREASE-KEY(H,x,k) and DELETE(H,x) receive a valid node handle. Finding x by an arbitrary key may require scanning the whole heap and is excluded from those operation bounds. Equal keys may coexist as distinct nodes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Separate the root list, child lists, and minimum pointer



Roots are 4,6,10. Node 4 owns 9 and 7; 7 owns 12. Root-list entry and minimum pointer have different roles. Each sibling group has its own circular list.

Node fields and the exact meaning of mark

A node stores key, parent, child, left, right, degree, and mark. The child pointer selects any child; siblings form a circular doubly linked list. A lone child points left and right to itself. H.roots enters the root list, H.min selects a minimum root, and H.n counts all nodes. Both entries are NIL in an empty heap.

A nonroot mark records one child loss since its current attachment. A second loss cuts the node. Our convention clears marks whenever nodes become roots or are linked as children; every root is unmarked.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circular lists: the mechanism behind constant-time meld

To remove a known node x, connect x.left.right=x.right and x.right.left=x.left. Replace the list entry if it names x; a singleton leaves NIL. Finally isolate x by setting both sibling pointers to itself. Only O(1) pointers change.

For nonempty circles entered at a and b, save al=a.left and bl=b.left. Set al.right=b, b.left=al, bl.right=a, and a.left=bl. Empty-list cases simply choose the other entry.

Below ATTACH-ROOT(H,x) takes an isolated node, splices it into the root list, and sets parent=NIL and mark=false. REMOVE-CHILD(p,x) repairs p.child, sibling links, and degree. These primitives neither search by key nor traverse every sibling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

INSERT, MINIMUM, and destructive UNION

Operation	Complete action	Actual cost
MAKE	roots=min=NIL; n=0	O(1)
INSERT(x)	New singleton root; update min; increment n	O(1)
MINIMUM	Return min, or NIL if empty	O(1)
UNION(H1,H2)	Splice root circles, choose the smaller non-NIL min, add counts	O(1)

UNION consumes two disjoint heaps. The result owns all their nodes; the old entries must not be used independently. H1=H2 is forbidden. Two empty inputs produce an empty result. This bound excludes rewriting a per-node heap-owner field; an implementation demanding that work cannot claim constant-time meld.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

EXTRACT-MIN: promote children, then consolidate

```
EXTRACT-MIN(H):
    z=H.min
    if z==NIL: return NIL
    while z.child!=NIL:
        c=z.child
        REMOVE-CHILD(z,c)
        ATTACH-ROOT(H,c)
    REMOVE-ROOT(H,z)
    H.n-=1
    if H.roots==NIL: H.min=NIL
    else: CONSOLIDATE(H)
    return z
```

Repeatedly remove the current child rather than following sibling pointers invalidated by mutation. Promotion clears parent and mark; isolate z before returning it. One original root does not imply an empty result: its promoted children may remain.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

CONSOLIDATE: leave at most one root of each degree

```
CONSOLIDATE(H):
    A=empty degree table; H.min=NIL
    while H.roots!=NIL:
        x=POP-ROOT(H)              // isolate one root
        d=x.degree
        while A[d]!=NIL:           // absent slot means NIL
            y=A[d]; A[d]=NIL
            if y.key<x.key: swap(x,y)
            LINK-ISOLATED(y,x)      // y.parent=x; y.mark=false
            d=x.degree
        A[d]=x
    for each non-NIL x in A:
        ATTACH-ROOT(H,x)
        if H.min==NIL or x.key<H.min.key: H.min=x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LINK and the degree-table contract

LINK-ISOLATED joins equal-degree isolated roots: the larger-key y becomes a child of x, increasing x.degree by one. Ties choose either parent. A can be a growing array using O(D(n)+1) slots. Draining the root list avoids revisiting old roots that have already been linked.

```
LINK-ISOLATED(y,x):
    require x!=y; x.degree==y.degree
    require x.key<=y.key; both roots isolated
    y.parent=x; y.mark=false
    splice y into x.child circular list
    x.degree+=1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Consolidation invariant and termination

At each outer-loop entry, unprocessed roots remain in H.roots; processed nodes lie in disjoint subtrees named by A. Each occupied slot stores one tree whose degree equals the index. Every original node belongs to exactly one of these parts.

The inner loop maintains d=x.degree. Linking an equal-degree y preserves the node set and heap order, replaces two roots by one, and increments d. Every link decreases the finite forest root count, so the loop terminates.

Each outer iteration removes an unprocessed root. At termination A represents all remaining trees, with distinct root degrees. Rebuilding the root list and comparing its roots gives the correct minimum. Distinct degrees are guaranteed only immediately after consolidation; INSERT, UNION, and CUT may create duplicates again.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

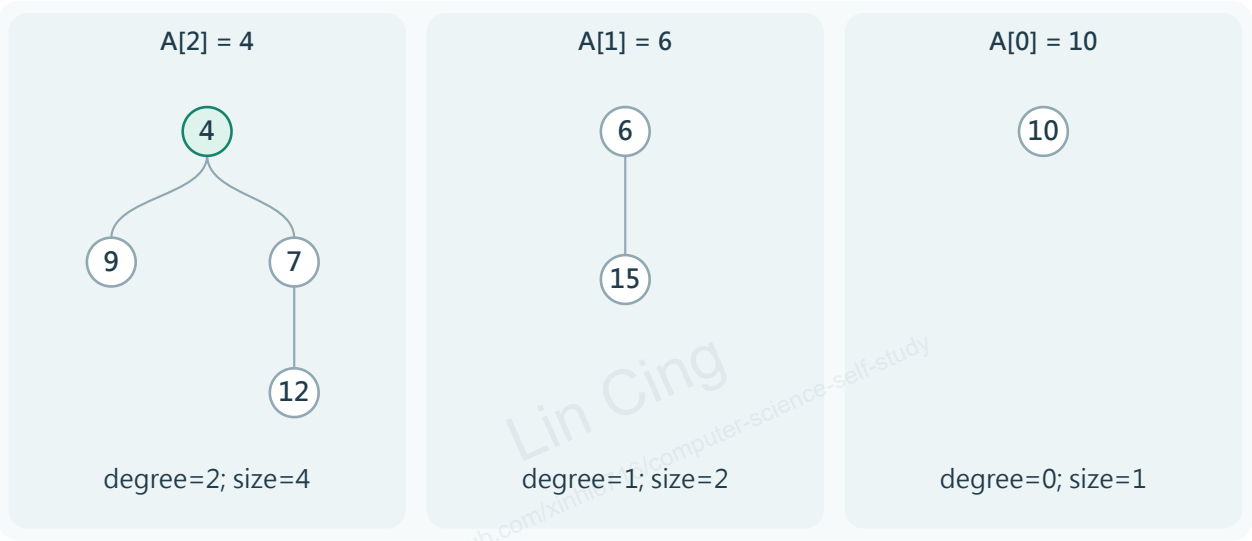
A worked consolidation: follow every degree collision

Start with eight singleton roots 1,4,9,7,12,6,15,10. Extract 1, then process the remaining roots in order 4,9,7,12,6,15,10; each starts with degree 0.

Root taken	Links in this iteration	Occupied A slots (degree:root)
4	None	0:4
9	Attach 9 under 4	1:4
7	None	0:7, 1:4
12	Attach 12 under 7; then 7 under 4	2:4
6	None	0:6, 2:4
15	Attach 15 under 6	1:6, 2:4
10	None	0:10, 1:6, 2:4

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Verify the final forest, degrees, and root count



Root degrees are 2,1,0. Subtree sizes total 4+2+1=7 and min=4. Other roots need not be ordered by key.

The same extraction: fewer roots release stored potential

Root 4 has children 9 and 7; node 7 retains child 12. Root 6 has child 15, and root 10 has no children. Their degrees are 2,1,0, with 4+2+1=7 nodes, and every parent key is no greater than its child key.

Stage	Roots r	Marks m	Potential K(r+2 m)
Before extracting 1: eight singleton roots	8	0	8 K
After removing 1 and completing four links	3	0	3 K

Potential falls by 5 K: removing the minimum eliminates one root and the four links eliminate four more. This connects the example to the amortized formula. **5 K is released potential, not an exact count of five machine instructions.**

These happen to be uncut binomial trees. After CUT, degree d no longer guarantees 2^d nodes; the general lower bound will be F_{d+2}.

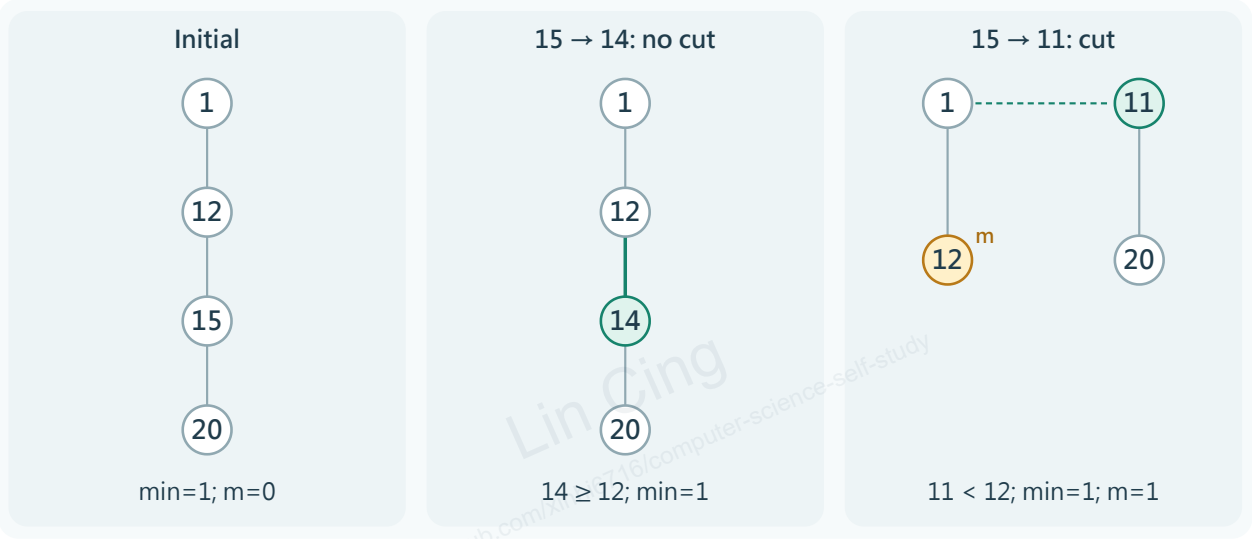
DECREASE-KEY: cut only when parent order is violated

```
DECREASE-KEY(H,x,k):
    if k>x.key: error
    x.key=k; p=x.parent
    if p!=NIL and x.key<p.key:
        CUT(H,x,p)
        CASCADING-CUT(H,p)
    if x.key<H.min.key: H.min=x

CUT(H,x,p):
    REMOVE-CHILD(p,x)       // degree decreases by one
    ATTACH-ROOT(H,x)       // parent=NIL; mark=false
```

Lowering x cannot violate its order relative to its children; only parent→x may fail. A root or a new key still at least its parent key needs no cut. The new minimum is either the old minimum or x, since no other key changed.

Worked decrease: 15→14 stays; 15→11 cuts the subtree



Start both cases from the left tree. Key 14 stays; key 11 is cut with child 20 still attached. Nonroot 12 records its first child loss.

Check both decreases field by field

Root 1 has child 12, which has child 15, which has child 20. Start each case from that same tree. Decreasing 15→14 keeps 14≥12 and changes no links. Decreasing 15→11 violates 11<12 and cuts the **entire subtree**: 20 remains a child of the node now keyed 11.

In the second case, an initially unmarked nonroot 12 becomes marked after its first child loss. Root 1 is unaffected. New root 11 exceeds the old minimum 1, so min stays. Decreasing an existing root, such as 3→0, updates min without a cut.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

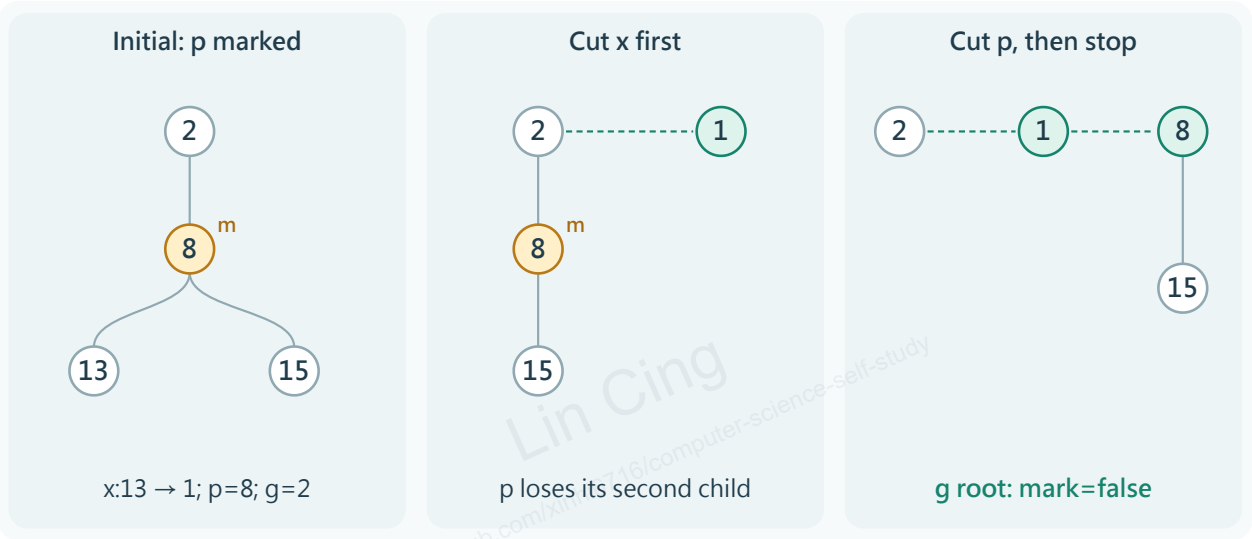
CASCADING-CUT: mark the first loss, cut on the second

```
CASCADING-CUT(H,y):
    while y.parent!=NIL:
        p=y.parent
        if y.mark==false:
            y.mark=true
            return
        CUT(H,y,p)
        y=p
    return                      // root: stop, do not mark
```

At entry y has just lost a child. If unmarked, record its first loss since this attachment and stop. If marked, this is its second loss: cut y, clear its mark, and continue at p, which has now lost y.

Deleting edges preserves heap order on all remaining edges. Cascading cuts limit how many children a nonroot can lose, preserving the relation between degree and subtree size; they do more than repair key order.

Cascading branch one: stop at a root grandparent



Cut x, then marked p. Stop at root g without marking it. Child 15 remains attached to p.

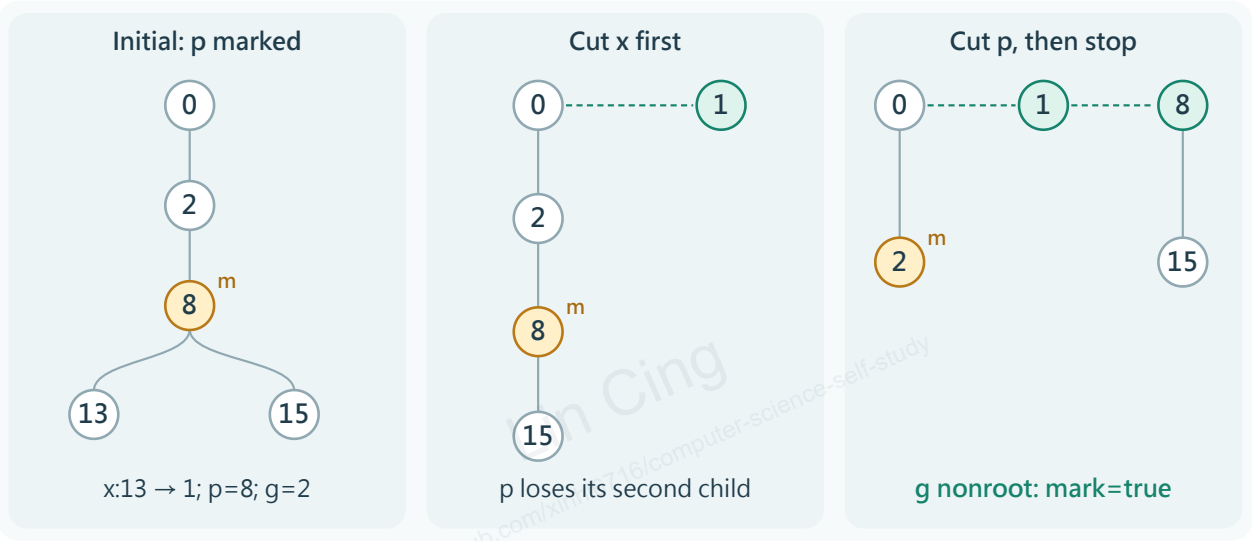
Root branch: root list and potential ledger

Initially the sole root is g=2. Its marked child p=8 has children x=13 and s=15. The mark records a previous child loss. Now decrease x from 13 to 1.

Cut x, then cut already-marked p. At g, parent=NIL, so stop and **do not mark root g**. Final roots are 2,1,8; node 8 still owns child 15. The minimum is 1.

With r roots and m marked nodes, (r,m) changes from (1,1) to (3,0). Unscaled potential r+2m changes from 3 to 3. Clearing p's mark offsets the two additional roots.

Cascading branch two: mark an unmarked nonroot grandparent



Here g has parent q and starts unmarked. After cutting x and p, mark g and stop. Root q=0 remains the minimum.

Nonroot branch: clear p, then mark g

Add root q=0 above the previous local structure, making g=2 an unmarked nonroot; p=8 remains marked. Decrease x=13 to 1. Cut x, then p. Now g.parent=q and g.mark=false, so mark g and stop.

Final roots are q=0, x=1, p=8. Node q retains g=2, and p retains s=15. The minimum stays 0. Roots rise 1→3, while marks stay 1→1: p is cleared and g is newly marked. Potential rises 3→5.

If g had already been marked, cut g as well and continue at q. **Parent and mark determine the endpoint**; marking the grandparent is not an unconditional outcome.

One potential pays for every operation

Choose one fixed constant K>0 and sum over all live, pairwise disjoint heaps:

$$\Phi = K \sum_{H} \big(r(H) + 2m(H)\big), \qquad \hat{c}_i = c_i + \Phi_i - \Phi_{i-1}.$$

Choose K once, sufficiently large to cover the constant-time pointer primitives below. Do not change it between operations. Starting empty gives $\Phi_0=0$ and nonnegative final potential, so $\Sigma c_i \le \Sigma \hat{c}_i$. A preexisting forest adds initial Φ_0 to the sequence bound.

INSERT adds one root and no mark, giving $\Delta\Phi=K$ and O(1) amortized cost. MINIMUM and MAKE leave potential unchanged. UNION replaces two heaps by their disjoint union; root and mark counts add, so global potential is unchanged and both actual and amortized meld costs are O(1).

Why a long cascade has constant amortized cost

With no cut, actual work and potential change are O(1). Otherwise let $c \ge 1$ count every cut, including the first cut of x. Each of the remaining $c-1$ cuts clears a marked nonroot. At most one final nonroot becomes newly marked; any old mark on x is also cleared. Hence

$$\Delta r = c, \qquad \Delta m \le 1 - (c-1) = 2 - c,$$
$$\Delta\Phi \le K(4-c).$$

For a fixed constant a, actual work is at most $a(c+1)$. Choose $K \ge a$:

$$\hat{c} \le a(c+1) + K(4-c) \le a + 4K = O(1).$$

This holds for every legal operation sequence without a random-input assumption. One actual cascade can still follow a long path and take linear time; O(1) is the amortized bound.

Extract-min analysis: many roots release stored potential

Before extraction let n be the node count, r the root count, and D=D(n) a maximum-degree bound. Promoting the removed root's children leaves R≤r−1+D roots. Every link removes a root, so there are at most R links. Child promotion, root processing, and degree-table initialization/rebuilding cost at most a(r+D+1).

After consolidation at most D+1 roots remain. Promotion and linking only clear marks, so

$$\Delta\Phi \le K(D+1-r).$$

$$\hat{c} \le a(r+D+1) + K(D+1-r) \le (a+K)(D+1).$$

Use the same K≥a; empty and singleton heaps cost O(1). This proves O(D+1), not yet logarithmic time: the degree theorem must supply D=O(log(n+1)). After n singleton insertions, the first extraction can take Θ(n) actual time.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The degree lemma: each retained child loses at most one child

Let x currently have d children $y_1, \ldots, y_d$, ordered by their **most recent attachment times** to x. For i≥2, all earlier children that still survive were already attached when y_i joined. Thus x then had degree at least i−1.

LINK joins equal-degree roots, so y_i also had degree at least i−1 at attachment. Losing a second child would have cut it from x. Since y_i still remains, it has lost at most one child:

$$\deg(y_1) \ge 0, \qquad \deg(y_i) \ge i-2 \quad (2 \le i \le d).$$

Current sibling degrees need not be sorted. Nor is a nonroot limited to one child loss over its entire lifetime: cutting and later relinking resets the relevant history.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

From child degrees to a Fibonacci subtree-size bound

Use strong induction on subtree node count to prove size(x)$\geq F_{d+2}$ for every node of degree d. Child subtrees are smaller, so their induction hypotheses apply. Combine the preceding degree lemma with monotonicity of Fibonacci numbers.

$$F_0 = 0, \quad F_1 = 1, \quad F_j = F_{j-1} + F_{j-2} \ (j \geq 2).$$

For d=0, size$\geq$1=F_2. For d=1, x and one child give size$\geq$2=F_3. For d$\geq$2, the first child contributes at least one node and the remaining children give

$$\text{size}(x) \geq 2 + \sum_{i=2}^{d} F_i = F_{d+2}.$$

The identity follows from $\sum_{i=0}^{d} F_i$=F_{d+2}−1: at d=0 it is 0=F_2−1; adding F_{d+1} and using the recurrence proves the next case. Inducting on subtree size avoids assuming an unproved monotonicity of minimum subtree size by degree.

Finish the degree bound: exponential size implies logarithmic degree

Let ϕ=(1+$\sqrt{5}$)/2, so ϕ^2=ϕ+1. Prove $F_{d+2}\geq\varphi^d$: the bases are 1$\geq$1 and 2$\geq$ϕ. For d$\geq$2,

$$F_{d+2} = F_{d+1} + F_d \geq \varphi^{d-1} + \varphi^{d-2} = \varphi^d.$$

For any node in a nonempty n-node heap, n$\geq$size(x)$\geq F_{d+2}\geq\varphi^d$, giving $d \leq \lfloor \log_\varphi n \rfloor$. Maximum degree, degree-table size, the post-consolidation root count, and amortized EXTRACT-MIN cost are therefore O(log(n+1)).

Degree d	0	1	2	3	4	5	6
Guaranteed minimum size F_{d+2}	1	2	3	5	8	13	21

This is a guaranteed lower bound, not a claim that a specified operation sequence attains equality for every d. Handle the empty heap separately rather than taking log 0.

DELETE: make the designated node uniquely minimum

DELETE(H,x) may first decrease x to a special sentinel below every ordinary comparison key, then call EXTRACT-MIN. For a nonroot, DECREASE-KEY correctly handles the parent's child loss and any cascade. Extraction promotes x's children and consolidates.

Writing “minus infinity” while allowing another node with the same comparison value is insufficient: min could still name the wrong node. Extend comparison to (tag,key), normally with tag=1; only the deletion target temporarily has tag=0. It becomes uniquely minimum. Restore its original key after returning the removed node.

Adding both operations under the same potential cancels the intermediate potential, giving amortized O(1)+O(log(n+1)). Direct internal-node removal is another possible implementation, but it must specify child promotion, parent marks, and cascading repair; promotion alone is incomplete.

Operation bounds and their conditions

Operation	Actual per-operation upper bound	Amortized upper bound
MAKE, MINIMUM, INSERT, UNION	O(1)	O(1)
DECREASE-KEY with a handle	O(n+1)	O(1)
EXTRACT-MIN, DELETE with a handle	O(n+1)	O(log(n+2))
SEARCH by an arbitrary key	O(n+1)	Heap order gives no automatic improvement

The logarithmic bound uses log(n+2) to retain constant cost when n=0. Valid-handle DELETE and DECREASE-KEY require a nonempty heap; empty EXTRACT-MIN is legal and returns NIL. Here n is the relevant heap size; UNION consumes disjoint input heaps. Bounds exclude allocator costs, per-node ownership relabeling, and locating a handle.

Turn operation counts into a graph-algorithm bound

A graph algorithm using O(V) insertions/extractions and O(E) handle-based decreases spends O(E+V log(V+1)) total heap time. Constants, memory usage, and access patterns still affect implementation choices; asymptotic bounds are not measured speed.

For V=1000 and E=8000, suppose each vertex is inserted/extracted at most once and each edge triggers at most one decrease:

Work	Maximum calls	Total Fibonacci-heap amortized bound
INSERT	1000	O(V)
EXTRACT-MIN	1000	O(V log(V+1))
DECREASE-KEY	8000	O(E)

Adding gives O(E+V log(V+1)); the corresponding binary-heap bound is O((E+V)log(V+1)). Counts are call limits, not milliseconds or exact costs: constants differ between representations.

Use integer bits to change the search model

A red-black tree compares keys; a **van Emde Boas tree (vEB)** also uses their bit representation. Fix $S \subseteq \{0, \ldots, u-1\}$ with $u = 2^m$, $m \geq 1$, and distinct keys. **n=|S| and universe size u are different parameters.**

Use a word RAM with keys and addresses fitting in one word. Shifts, masks, comparisons, and array indexing cost O(1); arbitrary-precision bit complexity is a different model.

Membership, updates, predecessor, and successor take worst-case $O(1 + \log\log u)$; extrema take O(1). Comparison-sorting lower bounds do not prohibit this: integer operations and initialization/storage costs are additional assumptions.

What direct addressing and naive recursion miss

A bit vector supports membership and updates in O(1), but scanning for the successor of 0 in $S = \{0, u-1\}$ can take Θ(u). A binary summary tree improves this query to O(log u).

Recursively storing roughly $\sqrt{u}$ clusters and their nonempty indices is not sufficient. Finding a cluster and then recursively finding its minimum gives

$$T(u) = 2T(\sqrt{u}) + O(1), \quad R(m) = 2R(m/2) + O(1) = \Theta(m).$$

The result is still Θ(log u). The crucial requirement is **at most one nonconstant recursive call per level**. Cached extrema make that possible.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Odd bit widths need unequal high and low parts

Let $\ell = \lfloor m/2 \rfloor$, $h = \lceil m/2 \rceil$, $L = 2^{\ell}$, and $H = 2^{h}$, so u=HL. For u>2:

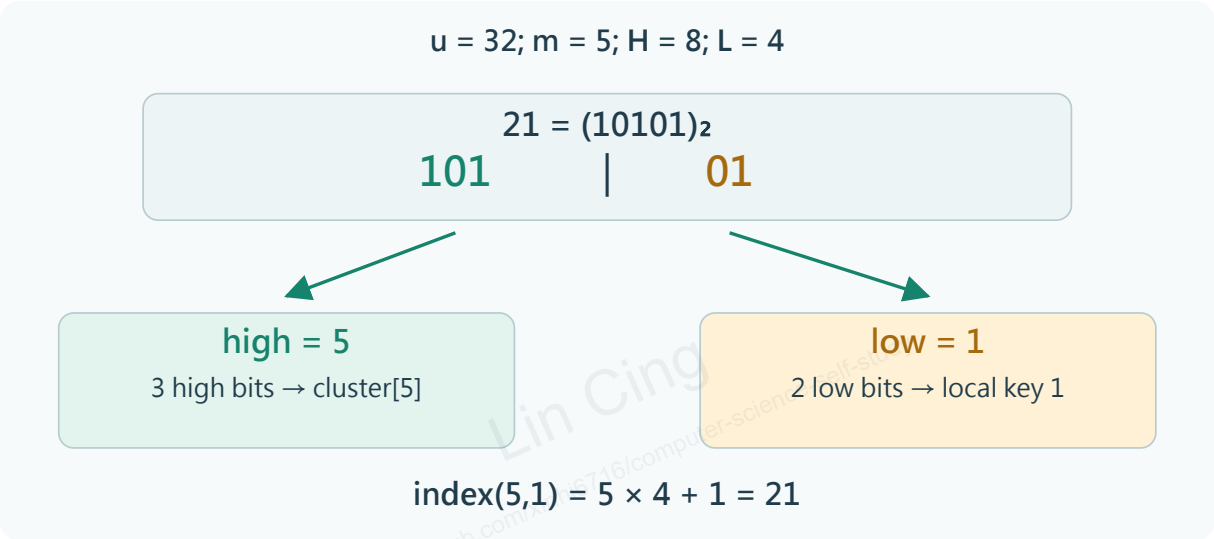
$$\text{high}(x) = \lfloor x/L \rfloor, \quad \text{low}(x) = x \bmod L, \quad \text{index}(i, j) = iL + j.$$

There are H clusters, each with universe L. The summary stores cluster indices and has universe H. **Recompute the split from the local u at every level.**

For u=32: m=5, h=3, ℓ=2, H=8, L=4. Split $21 = (10101)_2$ as $101 \mid 01$: high=5, low=1, and 5×4+1=21. The u=2 base has no recursive structures.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two coordinates of the same key



high selects a cluster; low is the key passed into it. A 5 in the summary means cluster 5 is nonempty, not that the original set contains key 5.

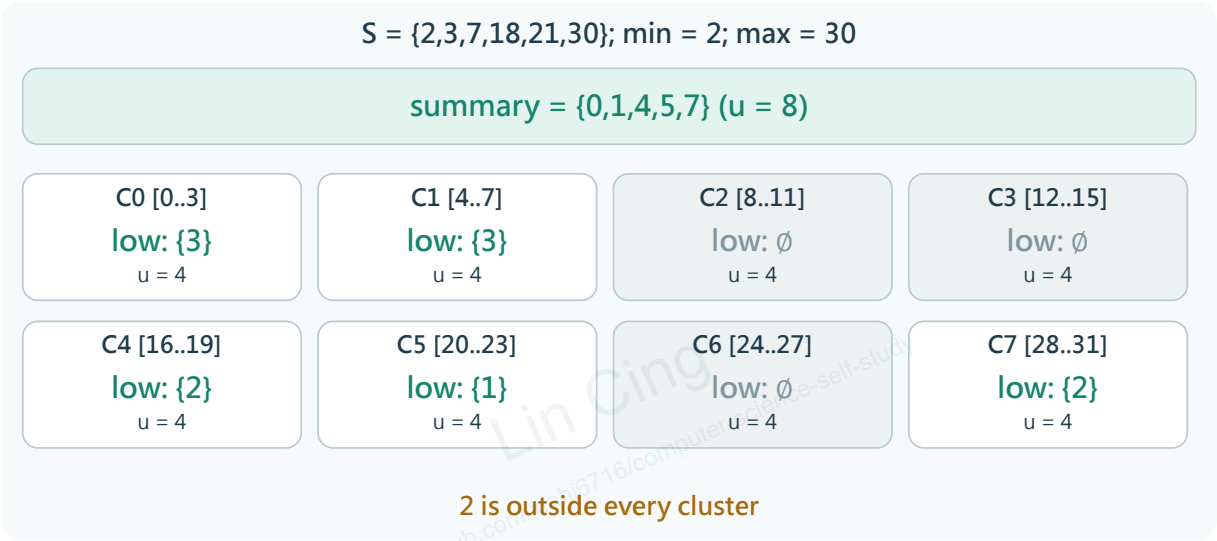
Representation invariant: the minimum stays outside clusters

Each node stores u, min, and max; for u>2 it also owns a summary and cluster[0..H−1]. Use a fully preallocated structure. Empty nodes have min=max=NIL.

For a nonempty set, min is stored **outside all clusters**. Every other key is represented by its low part in the cluster selected by its high part. A singleton has empty clusters and summary. With at least two keys, max also appears in a cluster.

The summary contains exactly the nonempty cluster indices: $i \in \text{summary} \iff C_i.\min \neq \text{NIL}$. Thus min=max detects a singleton in O(1), and inserting into an empty node sets min=max=x without descending.

A u=32 set: distinguish original keys, offsets, and summary



The set is {2,3,7,18,21,30}; top-level min=2 and max=30. C_0 stores only offset 3, not 2. The summary is {0,1,4,5,7}. Only the top level is expanded; every child obeys the same invariant.

Membership and valid arguments

```
MEMBER(V,x):                 # require 0<=x<V.u
    if V.min==NIL: return false
    if x==V.min or x==V.max: return true
    if V.u==2: return false
    return MEMBER(V.cluster[high(x)],low(x))
MINIMUM(V): return V.min
MAXIMUM(V): return V.max
```

high/low use V's local dimensions. Membership follows one cluster; extrema can stop the search early. NIL is a distinct state, never the valid key 0.

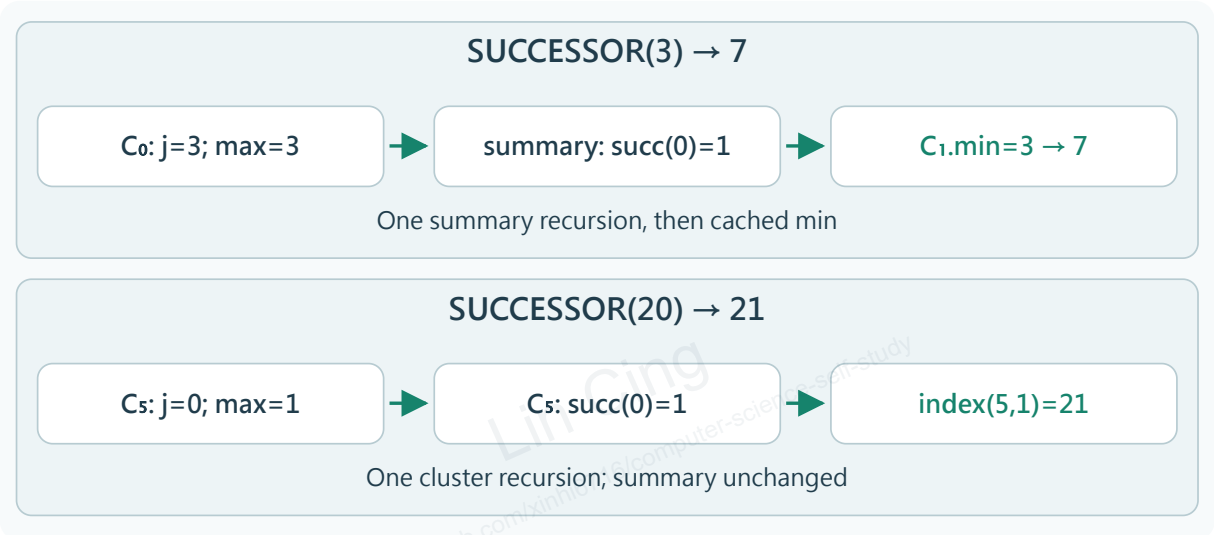
Successor is the smallest stored key **strictly greater than x**; predecessor is the largest strictly smaller key. Queries need not be members but must be inside the universe. Public interfaces reject invalid arguments before array access.

Successor: max chooses the recursive branch

```
SUCCESSOR(V,x):
    if V.min==NIL: return NIL
    if x<V.min: return V.min
    if x>=V.max: return NIL
    if V.u==2: return 1
    i=high(x); j=low(x); C=V.cluster[i]
    if C.max!=NIL and j<C.max:
        return index(i,SUCCESSOR(C,j))
    k=SUCCESSOR(V.summary,i)
    if k==NIL: return NIL
    return index(k,V.cluster[k].min)
```

If the current cluster's max exceeds j, recurse there. Otherwise skip directly to the summary. **Do not perform a failed recursive search first.** Reading the selected cluster's min then costs O(1).

Successor traces from one fixed state



Cached max chooses the cluster or summary branch; never both.

Check every successor coordinate

Use {2,3,7,18,21,30}. SUCCESSOR(3): i=0,j=3 and C_0.max=3, so query the summary. Its successor index is 1; C_1.min=3 gives 1×4+3=7.

SUCCESSOR(20): i=5,j=0 and C_5.max=1>0, so recurse only into C_5. Offset 1 reconstructs 21. Query 20 need not be stored. SUCCESSOR(30)=NIL because the order is strict.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Predecessor must include the detached minimum

```
PREDECESSOR(V,x):
    if V.min==NIL: return NIL
    if x>V.max: return V.max
    if x<=V.min: return NIL
    if V.u==2: return 0
    i=high(x); j=low(x); C=V.cluster[i]
    if C.min!=NIL and j>C.min:
        return index(i,PREDECESSOR(C,j))
    k=PREDECESSOR(V.summary,i)
    if k==NIL: return V.min
    return index(k,V.cluster[k].max)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

When k=NIL, return the detached min: earlier guards already established x>V.min, and no lower cluster exists. For example PREDECESSOR(3)=2 even though C_0 has no smaller offset. A symmetric copy of successor would miss 2.

Insert: swap out the old min and update only necessary summaries

```
INSERT-ABSENT(V,x):          # require x not in V
    if V.min==NIL: V.min=V.max=x; return
    if x<V.min: swap(x,V.min)
    if V.u>2:
        i=high(x); j=low(x); C=V.cluster[i]
        if C.min==NIL:
            INSERT-ABSENT(V.summary,i)
            C.min=C.max=j
        else: INSERT-ABSENT(C,j)
    if x>V.max: V.max=x
```

A smaller new key becomes min; **the old min** is then inserted below. An empty cluster requires a recursive summary insertion but only O(1) singleton initialization. A nonempty cluster already has its index in the summary, so recurse only into that cluster. The expensive branches are exclusive.

Insert 1: keep the new min outside and place the old min in C_0

Insert 1 into the original {2,3,7,18,21,30}:

Step	State	Invariant
Swap	min=1; pending x=2	New min stays outside
Split	high(2)=0, low(2)=2	Top-level L=4
C_0	{3} becomes {2,3}	Local min=2, max=3
Summary	Still {0,1,4,5,7}	C_0 was already nonempty
Finish	max remains 30	Largest key unchanged

C_0 has u=4, so its own split uses L=2. Its old local min 3 moves to local cluster 1 as offset 1. Coordinate systems change at each recursive level.

Delete: handle singletons and u=2 first

DELETE-PRESENT requires that x exists. Handle these two cases before the next page's general procedure:

```
DELETE-PRESENT(V,x):
    if V.min==V.max:
        V.min=V.max=NIL; return
    if V.u==2:
        V.min=V.max=1-x; return
    DELETE-GENERAL(V,x)
```

The first branch is a nonempty singleton by contract; all its clusters were already empty. At u=2 with two keys, the set is {0,1}, so deleting x leaves 1−x.

Calling this procedure on an absent key can erase valid data. A public interface first checks MEMBER and returns NOT_FOUND on failure; the precondition must be explicit.

General deletion: promote the next min, then remove its cluster copy

```
DELETE-GENERAL(V,x):         # u>2; at least two keys
    if x==V.min:
        k=V.summary.min
        x=index(k,V.cluster[k].min); V.min=x
    i=high(x); j=low(x); C=V.cluster[i]
    DELETE-PRESENT(C,j)
    if C.min==NIL:
        DELETE-PRESENT(V.summary,i)
        if x==V.max:
            k=V.summary.max
            if k==NIL: V.max=V.min
            else: V.max=index(k,V.cluster[k].max)
    elif x==V.max:
        V.max=index(i,C.max)
```

Why deletion rebinds x to the promoted minimum

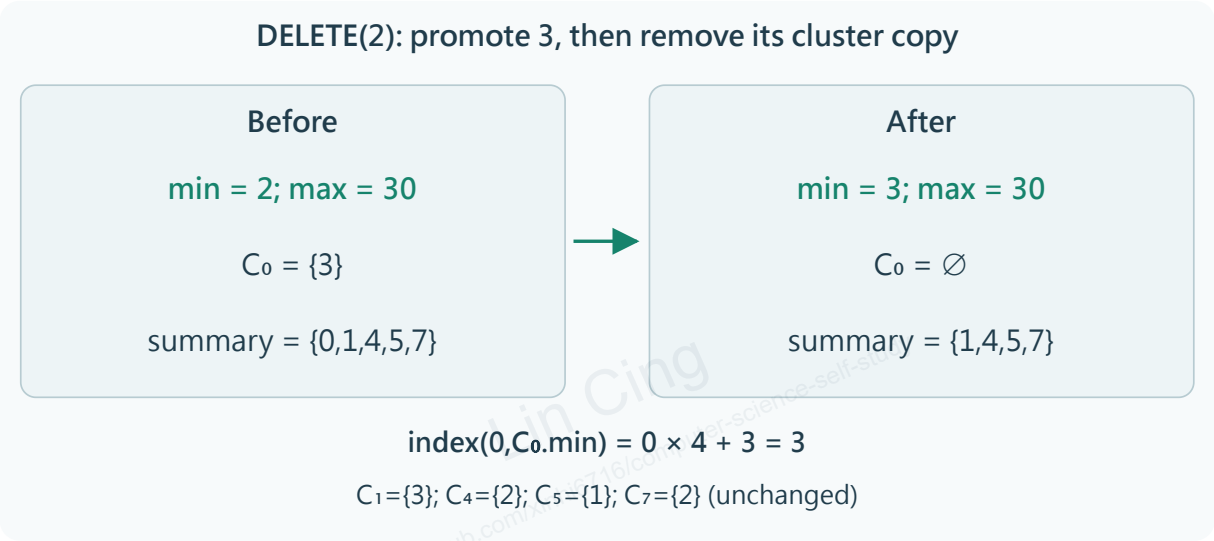
Promotion changes x to the new min. The recursive removal deletes its old cluster representation. Update the cluster and summary before reconstructing max; the previous largest cluster may no longer exist.

For {2,3}, delete 2: promotion sets x=3 and min=3; removing 3 empties C_0 and then the summary. Because the rebound x equals old max 3, set max=min=3, leaving the correct singleton.

With 7,18,21,30 also present, the rebound x=3 differs from max 30, so max stays unchanged. Every later comparison uses the current x, not the original argument 2.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Delete 2: promote 3 without shifting the other keys



Promote 3 to the top-level min, empty C_0, and remove summary index 0.

Check promotion and summary removal

Restart from the original set. summary.min=0 and C_0.min=3, so the new min is 3. Removing offset 3 empties C_0; deleting index 0 from the summary leaves {1,4,5,7}. max stays 30.

The promoted 3 now lives **only at the top level**. Keeping it in C_0 would violate the invariant. Here the cluster deletion removes a singleton and costs O(1); the summary is the only potentially expensive recursive path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deleting maxima and the final two elements

From the original set, delete 30,21,18,7,3,2 in order:

Removed	Remaining min / max	Top-level summary
30	2 / 21	{0,1,4,5}
21	2 / 18	{0,1,4}
18	2 / 7	{0,1}
7	2 / 3	{0}
3	2 / 2	Empty
2	NIL / NIL	Empty

After deleting 3, no cluster remains, so max becomes the detached min 2. Never use NIL from an empty summary as an array index. Deleting the final 2 clears extrema; all recursive structures are already empty.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Why two deletion calls still leave one expensive path

General deletion may call DELETE(cluster) and DELETE(summary), but the second occurs only if the cluster becomes empty. That means the cluster **was a singleton**, so its deletion took O(1).

Previous cluster	Cluster deletion	Remove summary index?
Singleton	O(1)	Yes; may recurse
At least two keys	May recurse	No; remains nonempty

Hence the cost is $\max\{T(L), T(H)\} + O(1)$, not $T(L) + T(H) + O(1)$. The representation and branch conditions prove a **worst-case** bound; neither probability nor amortization is needed.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Solve the recurrence in bit width

Let T(u) bound an operation and set $R(m) = T(2^m)$. One nonconstant recursive call reaches a child with at most $\lceil m/2 \rceil$ bits:

$$R(m) \le R(\lceil m/2 \rceil) + c, \qquad R(1) = O(1).$$

For m≥2, $\lceil m/2 \rceil \le 2m/3$, so at most $\lceil \log_{3/2} m \rceil$ levels reach the base. Constant work per level gives $O(1 + \log m) = O(1 + \log \log u)$.

Starting at m=5, a longest width path is 5→3→2→1; a cluster path can be 5→2→1. Odd widths do not have identical square-root child universes. The additive 1 handles the constant-time u=2 base.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Correctness through the represented set

Induct on bit width m. At m=1, enumerate the empty set, {0}, {1}, and {0,1}. Larger sets are uniquely reconstructed from the detached min and all cluster offsets.

Queries: extrema handle boundaries. If a same-cluster answer exists, no higher cluster can beat it; otherwise the summary selects the next nonempty cluster. Predecessor also considers detached min.

Insertion: swapping a new minimum with the old one relocates a key without losing it. Insert the pending key into its cluster and add a summary index exactly when that cluster first becomes nonempty.

Deletion: promote the least cluster key when deleting min, then remove its cluster copy; otherwise remove the requested key. Remove indices of newly empty clusters and reconstruct max from updated state. The induction hypothesis makes child operations correct, establishing the exact set change.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

An empty structure still costs Θ(u) space and initialization

The fully allocated version pays allocation before operations. For u>2, there are H clusters of universe L and a summary of universe H. Local pointer/field space is at most cH:

$$P(u) \leq HP(L) + P(H) + cH, \qquad P(2) \leq p_0.$$

Prove $P(u) \leq A(u-1) - B$ by strong induction. Choose B≥c and A≥B+p_0; substitution gives

$$P(u) \leq A[H(L-1) + (H-1)] - B(H+1) + cH \leq A(u-1) - B.$$

Thus space is O(u). Cluster-only base nodes already require Ω(u), giving Θ(u). Visiting and initializing all nodes also takes Θ(u). This proof permits unequal H and L.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Public interfaces, duplicate keys, and space tradeoffs

Case	Public contract
Duplicate INSERT(x)	MEMBER first; return ALREADY_PRESENT
Absent DELETE(x)	MEMBER first; return NOT_FOUND
x<0 or x≥u	Reject before indexing
Multiplicity needed	Maintain counts; change key set only on 0↔1
Satellite data	Specify key→record mapping and ownership

A membership check plus an update still costs O(1+log log u). Multiplicity assumes constant-time, nonoverflowing counters; duplicate keys must not enter INSERT-ABSENT directly.

The full tree is appropriate only when u is affordable. Lazy allocation and hash-table clusters require a new space/allocation/expected-time analysis; they do not inherit this deterministic worst-case proof. A balanced O(n)-space tree may be preferable for a sparse, huge universe.

Keep amortized, expected, and worst-case guarantees distinct

Structure / operation	Guarantee	Essential assumptions
Red-black query/update	Worst-case O(log(n+2))	Comparisons; O(n) space
Hash lookup	Expected O(1+α)	Stated hashing model; no ordered neighbors
Fibonacci decrease-key	Amortized O(1)	Valid handle; shared sequence potential
Full vEB predecessor/update	Worst-case O(1+log log u)	Integer word RAM; Θ(u) space/setup

Building a full vEB and performing q operations costs $O(u + q(1 + \log\log u))$. Initialization cannot disappear from the total. Comparison keys, integer universes, and randomized models impose different contracts; a context-free ranking of structures is misleading.

References and attribution

Cormen, Leiserson, Rivest, Stein. Introduction to Algorithms, 3rd ed. (2009), Chapters 10–14, 17–20. MIT 6.046J/18.401J (2004), L09: Balanced Search Trees, Piotr Indyk. MIT 6.046J/18.401J (2005), L08: Hashing II; L09: Randomly Built BSTs; L11: Augmenting Data Structures; L13: Amortized Analysis. Charles E. Leiserson and Erik D. Demaine. Lin Cing, personal handwritten data-structure notes: queues, search trees, red-black trees, multiway search trees, and heaps.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study